Protocole de comparaison des librairies d'analyse des trajectoires : TanaT

Garance Malnoë - Avril 2026

1. Introduction

L'objectif de ce document est de proposer une vue d'ensemble des possibilités qu'offre la librairie `TanaT` en Python pour l'analyse de séquences, et plus généralement de trajectoires. Ce document a été pensé pour être mis en parallèle avec son équivalent dédié aux packages `TraMineR` et `TraMineRextras` en R, afin de comparer les deux outils et de déterminer le plus adapté selon la situation à traiter.

Pour faciliter cette comparaison, les deux documents suivent le même plan :

- Présentation des formats de données acceptés par les packages.
- Exploration des jeux de données intégrés et méthodes de simulation.
- Introduction des données "fil rouge" utilisées pour illustrer la suite du document.
- Manipulation des données et sélection des séquences selon différents critères.
- Description statistique et visualisation des séquences.
- Construction de typologies (calcul des dissimilarités et algorithmes de clustering), incluant le cas multidimensionnel (*multichannel*).
- Autres analyses (divergence, survie, test entre groupes, formatage pour autres analyses)
- Autres possibilités (séquences d'événements et séquences d'intervalles).

Nous traiterons dans ce document essentiellement de l'analyse de séquences d'**états**. Les autres types (événements et intervalles) seront spécifiquement abordés dans la dernière section.

In [1]:
```
# Simulation
from tanat.dataset.simulation.sequence import (
    generate_event_sequences,
    generate_state_sequences,
    generate_interval_sequences,
)

# Définition objet séquence
from tanat.sequence import (
    EventSequencePool,
    IntervalSequencePool,
    StateSequencePool,
    EventSequenceSettings,
    StateSequenceSettings,
    IntervalSequenceSettings,
)
```

```
# Gestion de données / Critères de filtre
from tanat.criterion import (
    QueryCriterion,
    StaticCriterion,
    TimeCriterion,
    PatternCriterion,
    LengthCriterion,
    RankCriterion
)

# Métriques entités
from tanat.metric.entity import (
    HammingEntityMetric,
    HammingEntityMetricSettings,
)

# Métrique séquences
from tanat.metric.sequence import (
    EditSequenceMetric,
    EditSequenceMetricSettings,
    Chi2SequenceMetric,
    Chi2SequenceMetricSettings,
    DTWSequenceMetric,
    DTWSequenceMetricSettings,
    SoftDTWSequenceMetric,
    SoftDTWSequenceMetricSettings,
    LinearPairwiseSequenceMetric,
    LinearPairwiseSequenceMetricSettings,
    LCSSequenceMetric,
    LCSSequenceMetricSettings,
    LCPSequenceMetric,
    LCPSequenceMetricSettings
)

# Clustering
from tanat.clustering import (
    HierarchicalClusterer,
    HierarchicalClustererSettings,
)

# Trajectoires
from tanat.trajectory import TrajectoryPool

# Metriques trajectoires
from tanat.metric.trajectory import (
    AggregationTrajectoryMetric,
    AggregationTrajectoryMetricSettings,
)

# Visualisations
from tanat.visualization.sequence import SequenceVisualizer

# Analyse de survie
from tanat.survival import SurvivalAnalysis

import time
import pandas as pd
import numpy as np
import random
from datetime import datetime
```

```
import datetime as dt
from scipy.stats import entropy
from scipy.spatial.distance import pdist, squareform
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
```

2. Format de données

Un seul format de données est proposé par TanaT : un format **SPELL** (*vertical id-start-end-spells*) sans colonne de date de fin.

Prenons l'exemple avec deux séquences X et Y, où a, b et c représentent les états possibles (l'alphabet):

$$X = (a, a, a, b, b, c, c) \quad \text{et} \quad Y = (a, a, a, c, c, c, c)$$

In [2]:
```
df_exemple = pd.DataFrame(
    {
        "id":["X","X","X","Y","Y"],
        "begin":[1,4,6,1,4],
        "state":["a","b","c","a","c"]

    }
)
df_exemple
```

Out[2]:

	id	begin	state
0	X	1	a
1	X	4	b
2	X	6	c
3	Y	1	a
4	Y	4	c

La librairie `TanaT` est assez restrictive vis-à-vis du format accepté mais il est tout à fait possible d'utiliser des données sous d'autres formats en les formattant au préalable avec la librairie `pandas` .

3. Données exemples et simulations

La librairie `TanaT` met à disposition plusieurs jeux de données pour illustrer ses fonctionnalités :

Jeu de données	Type de séquence	Thématique	Individus	Longueur	États	Covariables

Jeu de données	Type de séquence	Thématique	Individus	Longueur	États	Covariables
MIMIC-IV (admissions)	intervalles et évènements	Registres médicaux	100	variable selon individus	9 ou 10	2
MIMIC-IV (procedures)	intervalles et évènements	Registres médicaux	92	variable selon individus	2 ou 352	2
MOOC-events	évènements	Utilisation d'un MOOC	130	variable selon individus	12 à 30 (différents niveaux de détail possible)	-
mvad	états	Transition école-travail	712	72	6	-

In [3]:
```
from tanat.dataset import access
mimic = access("mimic4")
mooc = access("mooc_events")
mvad = access("mvad")
```

Aussi, il est important de noter que `TanaT` ne propose pas de séquences d'états comportant plusieurs domaines (multichannel) ou des covariables.

La librairie offre également la possibilité de simuler ses propres données pour les trois types de séquences (intervalle, états et évènements) avec les fonction `generate_X_sequences()`. L'ajout de données manquantes est directement implémenté mais il n'est pas possible de préciser la proportion des différents types d'évènements, intervalles ou états.

In [4]:
```
# Séquences d'états
state_data = generate_state_sequences(
    n_seq= 10,
    seq_size=[4,5,3],
    vocabulary=[1,2,3],
    missing_data=0.1,
    entity_feature="event_type",
    seed=32,
)

# Séquences d'évènements
event_data = generate_event_sequences(
    n_seq= 10,
    seq_size=[4,5,3],
    vocabulary=[1,2,3],
    missing_data=0,
    entity_feature="event_type",
    seed=32,
)

# Séquences d'intervalles
interval_data = generate_state_sequences(
    n_seq= 10,
```

```
    seq_size=[4,5,3],
    vocabulary=[1,2,3],
    missing_data=0.2,
    entity_feature="event_type",
    seed=32,
)
```

4. Données fil-rouge

Pour la suite de cette présentation des possibilités de `TanaT`, nous utiliserons le jeu de données *Family life states from the Swiss Household Panel biographical survey* (`biofam`) pour les analyses portant sur des séquences simples (unidimensionnelles) et nous nous appuierons sur le jeu de données simulé *Synthetic Football Player Career Dataset* (`fb_careers`), mis en ligne sur Kaggle, pour les méthodes nécessitant des séquences multidimensionnelles (multichannel).

Family life states from the Swiss Household Panel biographical survey

Le jeu de données *Family life states* `biofam` est issu de l'enquête biographique rétrospective menée en 2002 par le Panel suisse de ménages. Il documente les trajectoires de formation familiale d'un échantillon de 2 000 individus sur une période de seize années, sans données manquantes, retraçant l'évolution de leur situation de l'âge de 15 ans à l'âge de 30 ans.

Ces séquences sont structurées au format STS et donc sont directement exploitable par `TraMineR`. L'alphabet des états se compose de huit statuts familiaux décrivant le lieu de résidence, le statut marital et la parentalité. Les individus sont ainsi catégorisés selon qu'ils vivent au domicile parental (*Parent*), qu'ils l'ont quitté (*Left*), qu'ils ont un enfant (*Child*), qu'ils ont quitté le domicile familial et qu'ils sont mariés (*Left+Marr*), qu'ils ont un enfant (*Child*), qu'ils ont quitté le domicile et ont un enfant (*Left+Child*), qu'ils soient hors du domicile familial, mariés et avec un enfant (*Left+Marr+Child*) ou qu'ils soient divorcés (*Divorced*).

En complément de la séquence d'état, le jeu de données inclut plusieurs covriables transversales : caractéristiques socio-démographiques de l'individu (sexe, année de naissance, première nationalité, langue de réponse au questionnaire) et culturelles (religion et participation religieuse), ainsi que sur l'origine sociale avec le statut social du père et de la mère. Cependant, certaines de ces variables sont manquantes chez certains individus.

Enfin, ce jeu de données propose deux sets de poids : l'un ajusté sur l'ensemble de la population suisse et l'autre ramené à la taille de l'échantillon. Cependant, ils ne sont fournis qu'à titre illustratif, puisque ce jeu de données est un sous-échantillon des réponses globales du panel.

Synthetic Football Player Career Dataset

Le jeu de données *Synthetic Football Player Career Dataset* `fb_careers` est un corpus de données simulées, mis à disposition sur la plateforme *Kaggle*. Il retrace les trajectoires sportives de 500 joueurs de football fictifs sur une période de treize années, couvrant les saisons de 2010/2011 à 2022/2023.

Cette base de donnée est proposé au format SPELL, compatible avec `TraMineR` . Elle rassemble plusieurs types d'informations sur le profil et les performances de chaque joueur avec des variables invariantes (nationalité, année de naissance, poste occupé sur le terrain) et des variables pouvant varier à chaque saison qui sont qualitatives (état de la carrière, niveau du club) quantitatives : nombres de matchs et de minutes jouées, buts, passes décisives, note, nombre de blessures subies, valeur marchande en milliers d'euros, et qualitatives : avancement dans la carrière (début ou *early*, apogée ou *peak*, et déclin ou *decline*) et le niveau du club dans lequel le joueur joue (1 à 5).

Dans le cadre de ce document, ces données simulées n'ont pas pour objectif de produire une véritable analyse. Elles constituent avant tout un support pour illustrer les possibilités des packages `TraMineR` et `TanaT` . Nous avons donc fait le choix de travailler sur les trajectoires formées par deux variables quantitatives que l'on a découpé en trois classes chacune (le nombre de but et le nombre de match), bien que dans la réalité cette analyse de soit pas forcément pertinente.

In [5]:
```
# Préparation du dataset biofam
biofam = pd.read_csv("../biofam.csv").dropna(axis=0,subset="id")
biofam_static = pd.read_csv("../biofam_sts.csv")

# Préparation du dataset fb_careers
fb_careers = pd.read_csv("../football_player_careers.csv")
fb_careers["begin"] = fb_careers["season"].apply(lambda x: x[0:4]).astype("Int64
k = 3
fb_careers["minutes_cat"] = pd.cut(fb_careers["minutes"],bins=k,labels=range(0,k
fb_careers["goals_cat"] = pd.cut(fb_careers["goals"],bins=k,labels=range(0,k))
fb_careers["matches_cat"] = pd.cut(fb_careers["matches"],bins=k,labels=range(0,k
fb_careers["rating_cat"] = pd.cut(fb_careers["rating"],bins=k,labels=range(0,k))
fb_careers["marketvalue_cat"] = pd.cut(fb_careers["market_value_m€"],bins=k,labe
simple_fb = fb_careers[["player_id","begin","birth_year","age","club_level","min
colname1 = "goals_cat"
colname2 = "matches_cat"
```

5. Gestion de données

Pour l'analyse de séquence il peut se réveler très utile de filtrer et sélectionner des séquences sur certains critères comme la sélection d'un échantillon ou le respect d'une contrainte de temporelle. La librairie Python \texttt{TanaT} sépare le filtrage et la sélection des données en plusieurs catégories :

- **Query** : sélection par requête sur les états (ex. : présence de l'état A).
- **Static** : filtrage via les covariables des individus (ex. : sélection par genre).
- **Time** : restriction temporelle ou sélection sur une fenêtre de temps donnée (ex. : trajectoires entre 2001 et 2005).

- **Pattern** : recherche de motifs séquentiels précis (ex. : état A → état B → état D).
- **Length** : filtrage selon la longueur des séquences (ex. : taille strictement supérieure à 5).

Nous allons illuster ces différents filtrages par des exemples sur le jeu de données `biofam` .

Nous commençons tout d'abord par définir l'objet séquence avec la fonction `StateSequencePool()` .

In [6]:
```
# Création de l'objet séquence pour biofam
# Dépliment de biofam en format lisible pour TanaT
biofam_expanded = biofam.copy()
biofam_expanded['timestep'] = biofam_expanded.apply(
    lambda row: list(range(int(row['begin']), int(row['end']) + 1)), axis=1
)
biofam_expanded = biofam_expanded.explode('timestep')
biofam_expanded["timestep"] = biofam_expanded["timestep"].astype("Int64")

biofam_settings = {
    "id_column": "id",
    "start_column": "timestep",  # On utilise la nouvelle colonne dépliée
    "default_end_value":16,
    "entity_features": ["states"],
}
biofam_pool = StateSequencePool(biofam_expanded, biofam_settings)
```

Avec la librairie `TanaT` , tous les filtrages fonctionnent de la même façon : tout d'abord par la création d'un crière (`QueryCriterion` , `FilterCriterion` , ...), puis par l'application à l'objet séquences la méthode `filter` en précisant le critère.

Query : Filtrer les séquences selon une requête

Exemple : Ne sélectionner que les séquences qui passent au moins une fois par l'état "L" (Left) dans leur séquence.

In [7]:
```
criterion = QueryCriterion(query="states == 'L'")
# Au niveau des séquences
filtered_seqpool = biofam_pool.filter(criterion, level="sequence")
```

Static : filtrer les séquences selon des covariables statiques

Exemple : Sélection des personnes née strtictement après 1950

In [8]:
```
# Ajout d'information statiques : sexe (sex) et année de naissance (birthyr)
static_df = biofam_static[["id","sex","birthyr"]]
biofam_pool = biofam_pool.add_static_features(static_df, override=True)
# Sélection des individus nés après 19k0
criterion = StaticCriterion(query="birthyr > 1950")
filtered_seqpool = biofam_pool.filter(criterion)
```

Time : sélectionner les séquences ayant des évènements se

déroulant dans une certaine fenêtre de temps ou ne garder que les entités se déroulant durant cette période

Exemple : sélectionner les séquences comprises (en entier) entre le temps 1 et le temps 8.

In [9]:
```
time_window_criterion = TimeCriterion(
    start_after=1,
    end_before=8,
    sequence_within=True,
)
filtered_seqpool = biofam_pool.filter(time_window_criterion, level="sequence")
```

```
No valid IDs found in sequence data. Returning empty sequence data.
No valid IDs found in static data. Returning empty static_data.
```

Dans `biofam`, toutes les séquences sont de taille 16, nous obtenons donc un objet séquence vide.

Pattern : filtrer les séquences selon la présence de pattern (ex. séquences avec etat A -> état B -> état D).

Exemple : Présence du pattern Parent -> Left + Married.

In [10]:
```
pattern_criterion = PatternCriterion(
    pattern={"states": ["P", "LM"]},
    contains=True,
)
filtered_seqpool = biofam_pool.filter(pattern_criterion, level="sequence")
```

Length : filtrer les séquences selon leur longueur

Exemple : Filtrer les séquences de taille strictement supérieure à 10

In [11]:
```
length_criterion = LengthCriterion(gt=10) # gt = greater
filtered_seqpool = biofam_pool.filter(length_criterion)
```

Changement de granularité : aggrégation des états

Dans le même esprit de préparation des données, il est parfois utile de procéder à un **changement de granularité**, par exemple dans un contexte où les différents individus n'auraient pas répondu à la même fréquence mais que l'on souhaite les étudier ensemble.

`TanaT` ne met à disposition de fonction ou de méthode pour le faire automatiquement mais il est possible avec `pandas` de modifier au préalable les données.

Exemple : aggrégation des états 4 à 4 en gardant l'état le + fréquent.

In [12]:
```
chunk_size = 4
biofam_expanded['agg_timestep'] = ((biofam_expanded['timestep'] - 1) // chunk_si
```

```
def etat_le_plus_frequent(series):
    modes = series.mode()
    return modes.iloc[0] if not modes.empty else None

# 3. Grouper par individu (id) et par bloc, puis appliquer la fonction
biofam_agg_long = (
    biofam_expanded
    .groupby(['id', 'agg_timestep'])['states']
    .agg(etat_le_plus_frequent)
    .reset_index()
)

biofam_settings_long = {
    "id_column": "id",
    "start_column": "agg_timestep",  # On utilise la nouvelle colonne dépliée
    "default_end_value":16,
    "entity_features": ["states"],
}
biofam_pool_long = StateSequencePool(biofam_expanded, biofam_settings)
```

6. Description des séquences (statistiques et visualisations)

La librairie `TanaT` met à disposition plusieurs outils statistiques et de visualisation pour explorer les séquences de manière globale et individuelle.

Alphabet

L'alphabet qui décrit les séquences s'obtient avec la méthode `vocabulary` de l'objet séquence.

In [13]:
```
biofam_pool.vocabulary
```

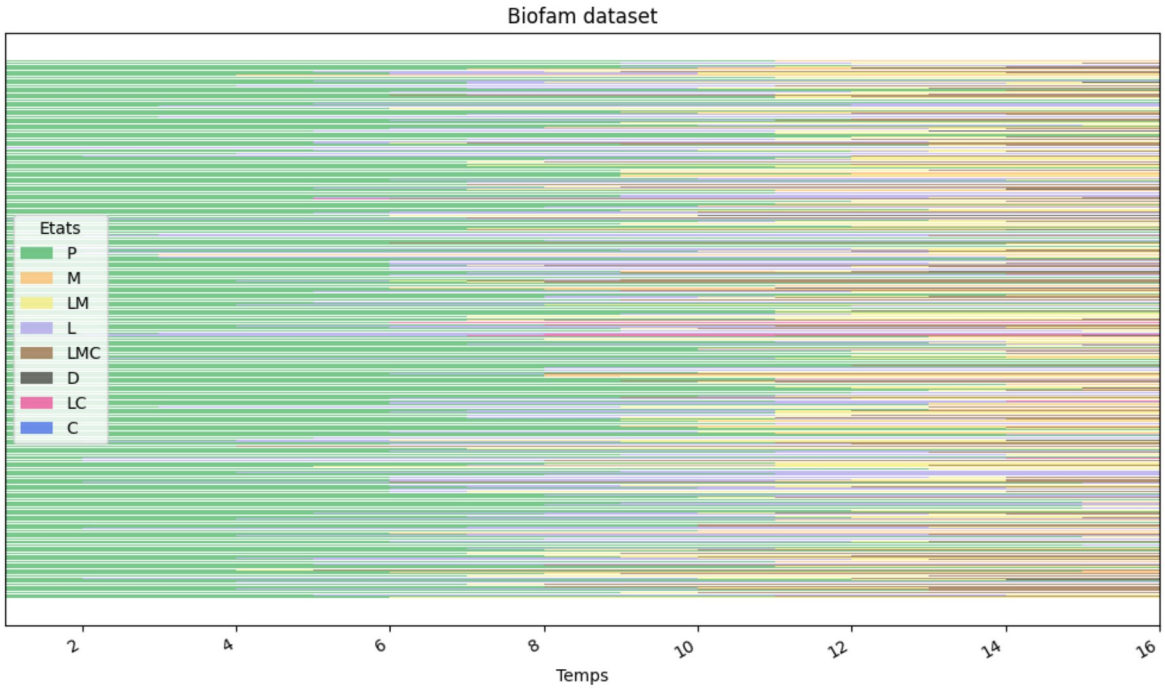
Out[13]:
```
{'C', 'D', 'L', 'LC', 'LM', 'LMC', 'M', 'P'}
```

Visualisation des séquences

La visualisation des trajectoires se fait avec l'objet `SequenceVisualizer` avec la méthode `timeline` qui s'adapte autant à la visualisations d'un grand nombre de séquences, qu'un nombre restreint.

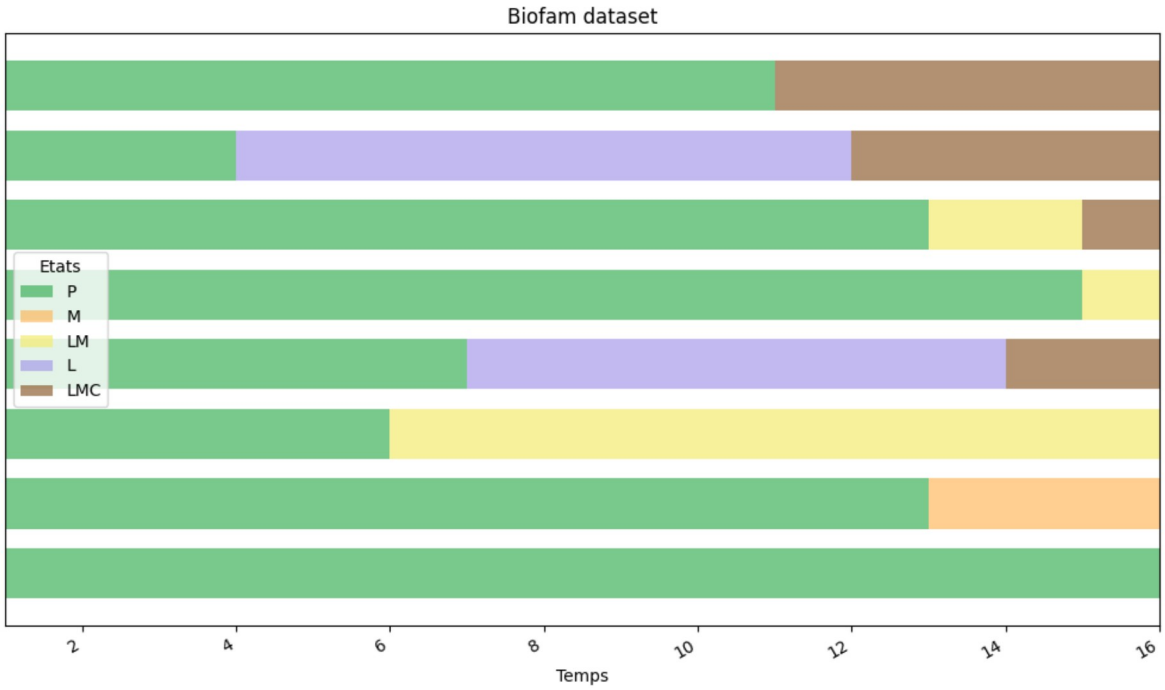
In [14]:
```
biofam_colors = {
    'P': "#40b15c",
    'L': "#a89cec",
    'M': "#ffbb62",
    'LM': "#f5ec73",
    'C':"#3760e9",
    'LC':"#ee4091",
    'LMC': "#916237",
    'D':"#363631",
}
```

In [15]:
```
SequenceVisualizer.timeline(stacking_mode="flat") \
    .title("Biofam dataset") \
    .colors(biofam_colors) \
    .marker(spacing=0.9) \
    .legend(show=True, title="Etats", loc="center left") \
    .x_axis(label="Temps") \
    .draw(biofam_pool)
```



Out[15]:

In [16]:
```
criterion = QueryCriterion(query="id <10")
filtered_seqpool = biofam_pool.filter(criterion, level="sequence")

SequenceVisualizer.timeline(stacking_mode="flat") \
    .title("Biofam dataset") \
    .colors(biofam_colors) \
    .marker(spacing=0.9) \
    .legend(show=True, title="Etats", loc="center left") \
    .x_axis(label="Temps") \
    .draw(filtered_seqpool)
```



Out[16]:

Fréquence des séquences

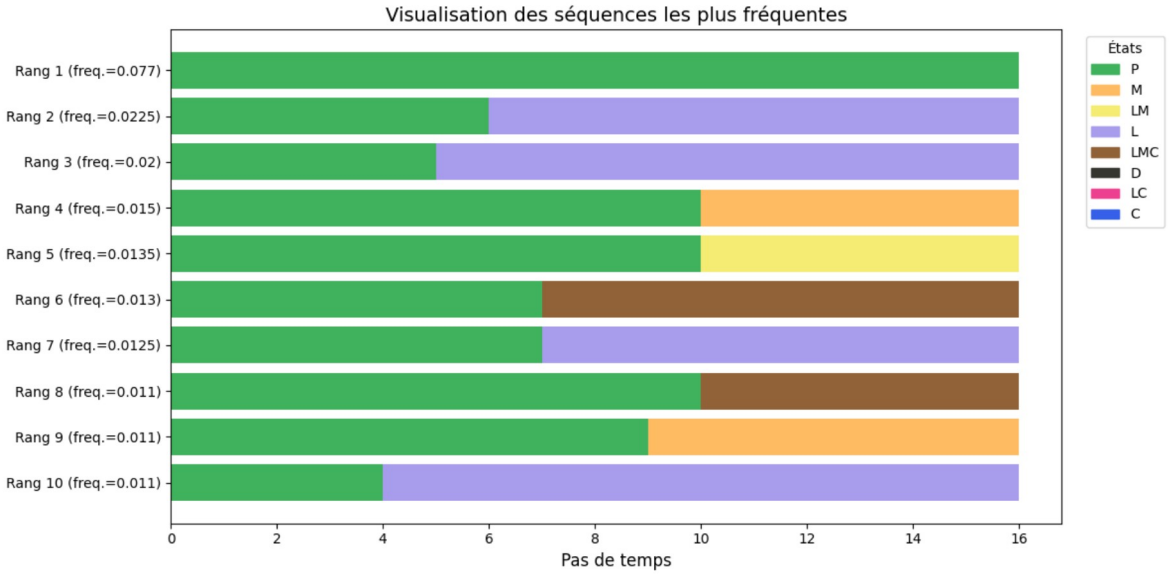
TanaT ne propose pas de méthode pour obtenur les fréquences des séquences mais il est possible des les calculer en utilisant pandas ainsique de produire une visualisation avec matplotlib .

In [17]:
```
# Fréquences des séquences entières avec pandas
biofam_expanded = biofam_expanded.sort_values(by=['id', 'timestep'])
full_sequences = biofam_expanded.groupby('id')['states'].apply(lambda x: '-'.joi
seq_freq = full_sequences.value_counts(normalize=True)
seq_freq
```

Out[17]:
```
states
P-P-P-P-P-P-P-P-P-P-P-P-P-P-P                    0.0770
P-P-P-P-P-L-L-L-L-L-L-L-L-L-L                    0.0225
P-P-P-P-P-L-L-L-L-L-L-L-L-L-L                    0.0200
P-P-P-P-P-P-P-P-P-P-M-M-M-M-M                    0.0150
P-P-P-P-P-P-P-P-P-P-LM-LM-LM-LM-LM-LM            0.0135
                                                  ...
P-P-P-P-P-P-P-P-L-L-LM-LM-LM-LMC-LMC-LMC         0.0005
P-P-P-P-P-L-L-LM-LM-LM-LMC-LMC-LMC-LMC-LMC-LMC   0.0005
P-P-P-P-L-L-L-L-L-L-L-LM-LM-LMC-LMC-LMC          0.0005
P-L-L-L-L-L-L-L-LM-LM-LM-LMC-LMC-LMC-LMC-LMC     0.0005
P-P-P-P-L-L-L-L-L-L-L-L-L-LM-LMC-LMC             0.0005
Name: proportion, Length: 537, dtype: float64
```

In [18]:
```
# Visualisation avec matplotlib
top_10_seqs = seq_freq.head(10)
top_10_patterns = top_10_seqs.index.str.split('-').tolist()
frequencies = top_10_seqs.values

## Palette couleurs homogènes
unique_states = biofam_expanded['states'].unique()
color_dict = biofam_colors
```

```
## Figure
fig, ax = plt.subplots(figsize=(12, 6))
## Tracer un bloc pour reprendre l'esthétique de SequenceVizualiser
for y_pos, (sequence, freq) in enumerate(zip(top_10_patterns, frequencies)):
    y = len(top_10_patterns) - 1 - y_pos
    for t, state in enumerate(sequence):
        ax.barh(y, width=1, left=t, height=0.8, color=color_dict[state], edgecol

## Légende, axes, esthétiques
ax.set_yticks(range(len(top_10_patterns)))
yticklabels = [f"Rang {i+1} (freq.={f})" for i, f in enumerate(frequencies)]
ax.set_yticklabels(yticklabels[::-1])
ax.set_xlabel("Pas de temps", fontsize=12)
ax.set_title("Visualisation des séquences les plus fréquentes", fontsize=14)
legend_patches = [mpatches.Patch(color=color_dict[state], label=state) for state
ax.legend(handles=legend_patches, title='États', bbox_to_anchor=(1.02, 1), loc='

plt.tight_layout()
plt.show()
```



Fréqence des états co-occurents

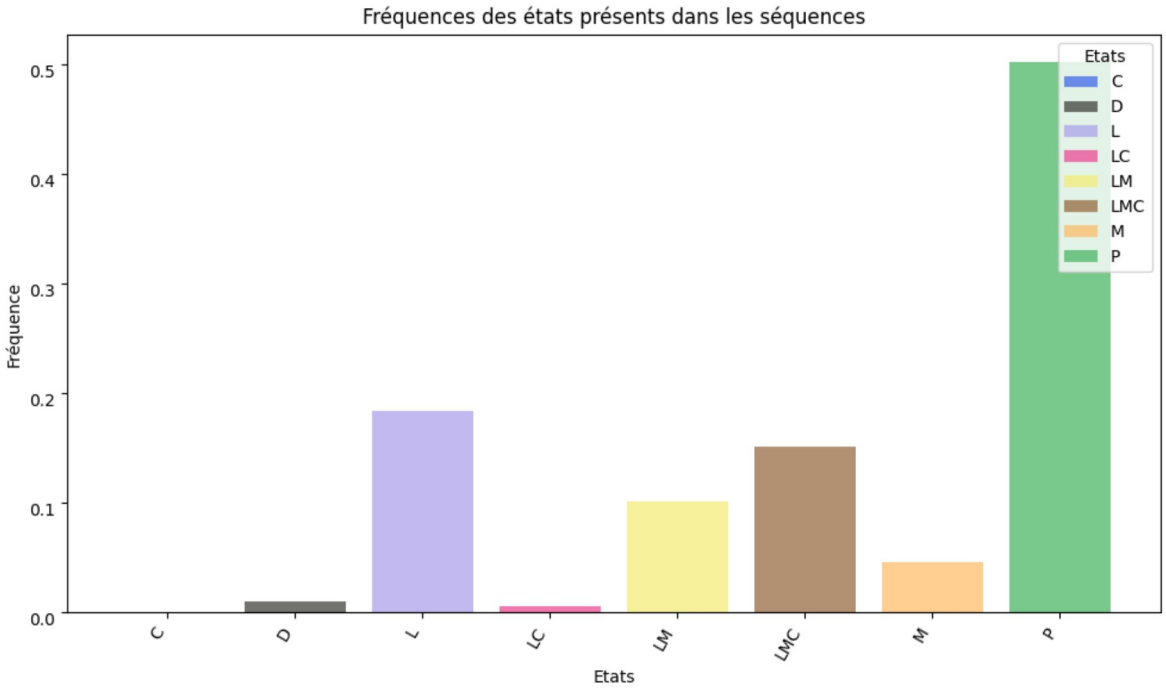
Fréquence des états

De même, il n'est pas possible d'extraire directement la fréquence des états directement avec la librairie mais il est possible de les obtenir avec `pandas` et de produire une visualisation sous forme de barplot avec `matplotlib` .

In [19]:
```
# Calcul de la fréquence des états avec pandas
state_counts = biofam_expanded.groupby(['states']).size()
state_freq = state_counts/sum(state_counts)
state_freq
```

Out[19]:
```
states
C      0.000594
D      0.010063
L      0.184000
LC     0.005219
LM     0.101375
LMC    0.151188
M      0.045812
P      0.501750
dtype: float64
```

In [20]:
```
SequenceVisualizer.histogram() \
    .title("Fréquences des états présents dans les séquences") \
    .colors(biofam_colors) \
    .legend(show=True, title="Etats", loc="upper right") \
    .x_axis(label="Etats") \
    .y_axis(label="Fréquence") \
    .draw(biofam_pool)
```



Out[20]:

Tableau des transitions

Pour analyser les dynamiques entre les états, il peut également être intéressant d'obtenir la matrice des transitions, que l'on peut construire avec `pandas`.

In [21]:
```
# Calcul de la table des transition avec pandas
## Réindexage pour crosstab
biofam_expanded_redindex = biofam_expanded.reset_index(drop=True)
biofam_expanded_redindex = biofam_expanded_redindex.sort_values(by=['id', 'times
## Création d'une colonne next_state pour garder l'état suivant et faire une cro
biofam_expanded_redindex['next_state'] = biofam_expanded_redindex.groupby('id')[
transitions_data = biofam_expanded_redindex.dropna(subset=['next_state'])
## Cross tabulation entre l'état actuel et l'état suivant pour tous les individu
transition_probs = pd.crosstab(
    transitions_data['states'],
    transitions_data['next_state'],
```

```
    normalize='index',
    rownames=['Etat t'],
    colnames=['Etat t+1']
)
transition_probs
```

Out[21]:

Etat t+1 / Etat t	C	D	L	LC	LM	LMC	M	P
C	0.812500	0.000000	0.000000	0.062500	0.000000	0.000000	0.125000	0.000000
D	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
L	0.000000	0.000178	0.889840	0.003922	0.083422	0.022638	0.000000	0.000000
LC	0.000000	0.000000	0.000000	0.881944	0.000000	0.118056	0.000000	0.000000
LM	0.000000	0.013598	0.000000	0.000000	0.786960	0.199442	0.000000	0.000000
LMC	0.000000	0.006068	0.000000	0.000000	0.000000	0.993932	0.000000	0.000000
M	0.000000	0.009531	0.000000	0.000000	0.010326	0.011120	0.969023	0.000000
P	0.000377	0.000000	0.054584	0.001069	0.031820	0.011131	0.015344	0.885675

Distribution transversale

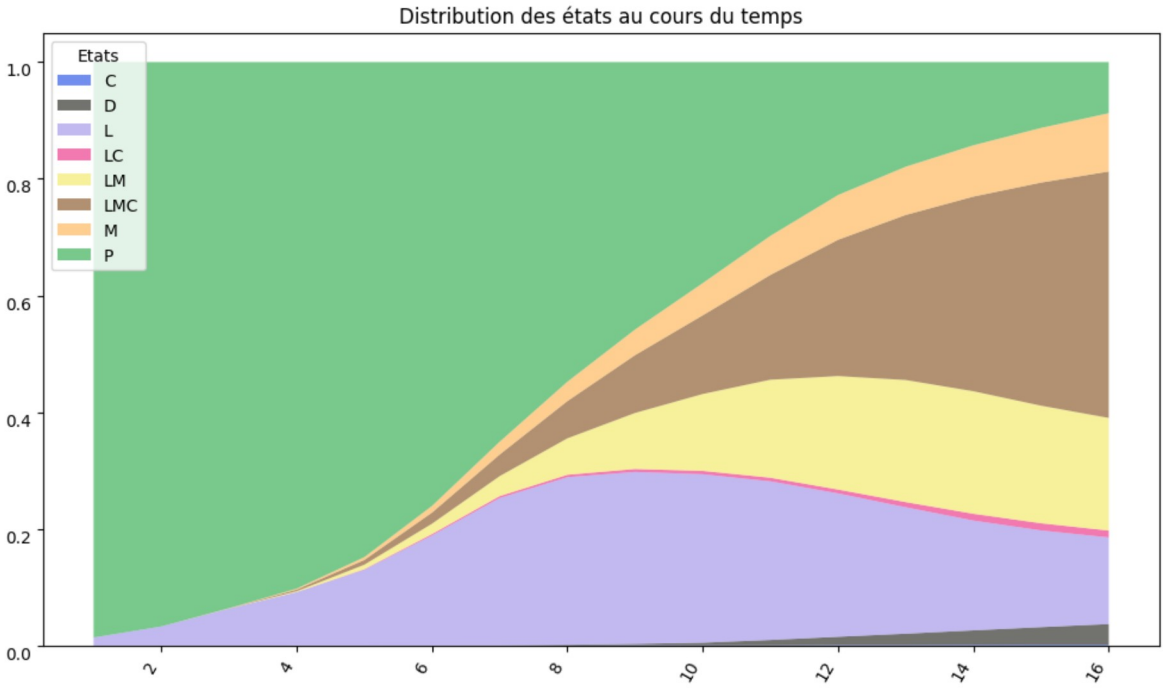
De même pour la visualisation de l'évolution de la distribution transversale, il est nécessaire de passer par `pandas` et `matplotlib` .

In [22]:
```
# Fréquence de chaque état à chaque point dans le temps avec pandas
state_counts = biofam_expanded.groupby(['timestep', 'states']).size().unstack(fi
state_probs = state_counts.div(state_counts.sum(axis=1), axis=0)
state_probs
```

Out[22]:

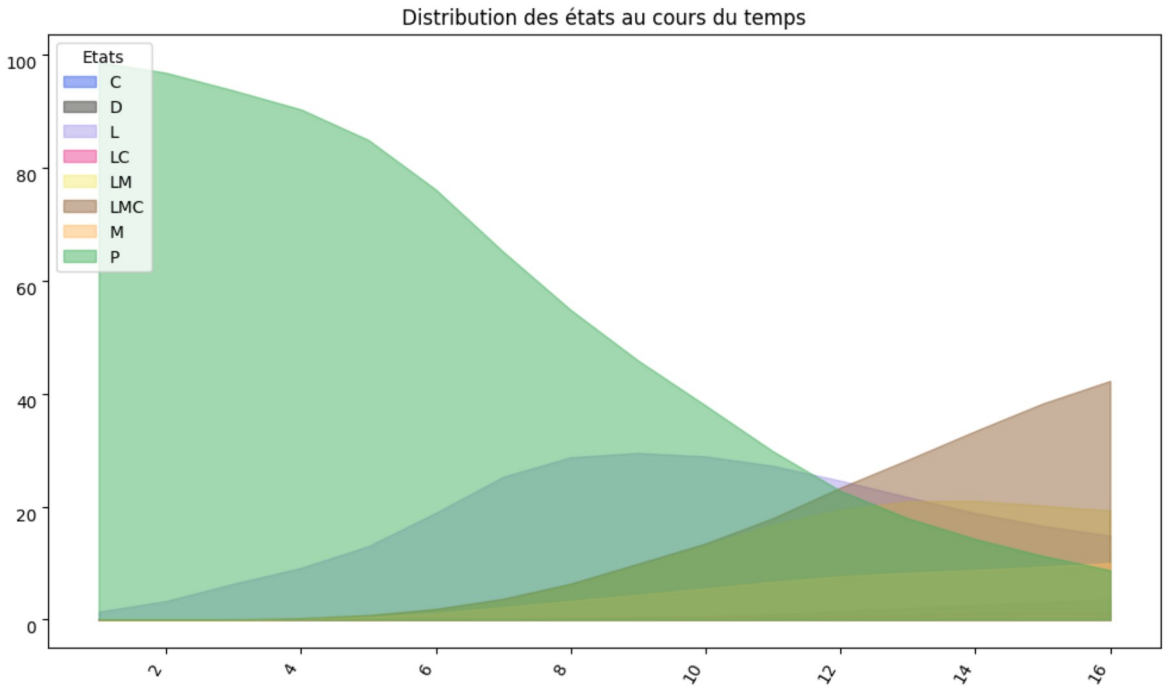
states	C	D	L	LC	LM	LMC	M	P
timestep								
1	0.0000	0.0000	0.0140	0.0000	0.0000	0.0000	0.0000	0.9860
2	0.0000	0.0000	0.0520	0.0000	0.0000	0.0000	0.0000	0.9480
3	0.0000	0.0000	0.0750	0.0000	0.0005	0.0005	0.0005	0.9235
4	0.0005	0.0000	0.1080	0.0000	0.0025	0.0050	0.0030	0.8810
5	0.0000	0.0000	0.1520	0.0015	0.0130	0.0110	0.0070	0.8155
6	0.0000	0.0000	0.2260	0.0030	0.0225	0.0270	0.0160	0.7055
7	0.0000	0.0005	0.2795	0.0035	0.0460	0.0470	0.0280	0.5955
8	0.0005	0.0020	0.2950	0.0045	0.0790	0.0805	0.0385	0.5000
9	0.0010	0.0025	0.2945	0.0055	0.1130	0.1175	0.0495	0.4165
10	0.0000	0.0065	0.2830	0.0060	0.1505	0.1520	0.0610	0.3410
11	0.0010	0.0120	0.2610	0.0060	0.1855	0.2070	0.0730	0.2545
12	0.0010	0.0165	0.2305	0.0070	0.2035	0.2600	0.0805	0.2010
13	0.0010	0.0225	0.2030	0.0105	0.2150	0.3055	0.0850	0.1575
14	0.0015	0.0275	0.1735	0.0125	0.2050	0.3615	0.0910	0.1275
15	0.0015	0.0330	0.1580	0.0120	0.1980	0.4030	0.0965	0.0980
16	0.0015	0.0380	0.1390	0.0115	0.1880	0.4415	0.1035	0.0770

In [23]:
```
SequenceVisualizer.distribution(
    distribution_type="proportion",
    granularity="UNIT") \
    .legend(show=True, title="Etats", loc="upper left") \
    .colors(biofam_colors) \
    .title("Distribution des états au cours du temps") \
    .draw(biofam_pool)
```



Out[23]:

In [24]:
```
SequenceVisualizer.distribution(
    granularity="UNIT",
    distribution_type="percentage",
    stacked=False) \
    .legend(show=True, title="Etats", loc="upper left") \
    .colors(biofam_colors) \
    .title("Distribution des états au cours du temps") \
    .draw(biofam_pool)
```



Out[24]:

Entropie transversale

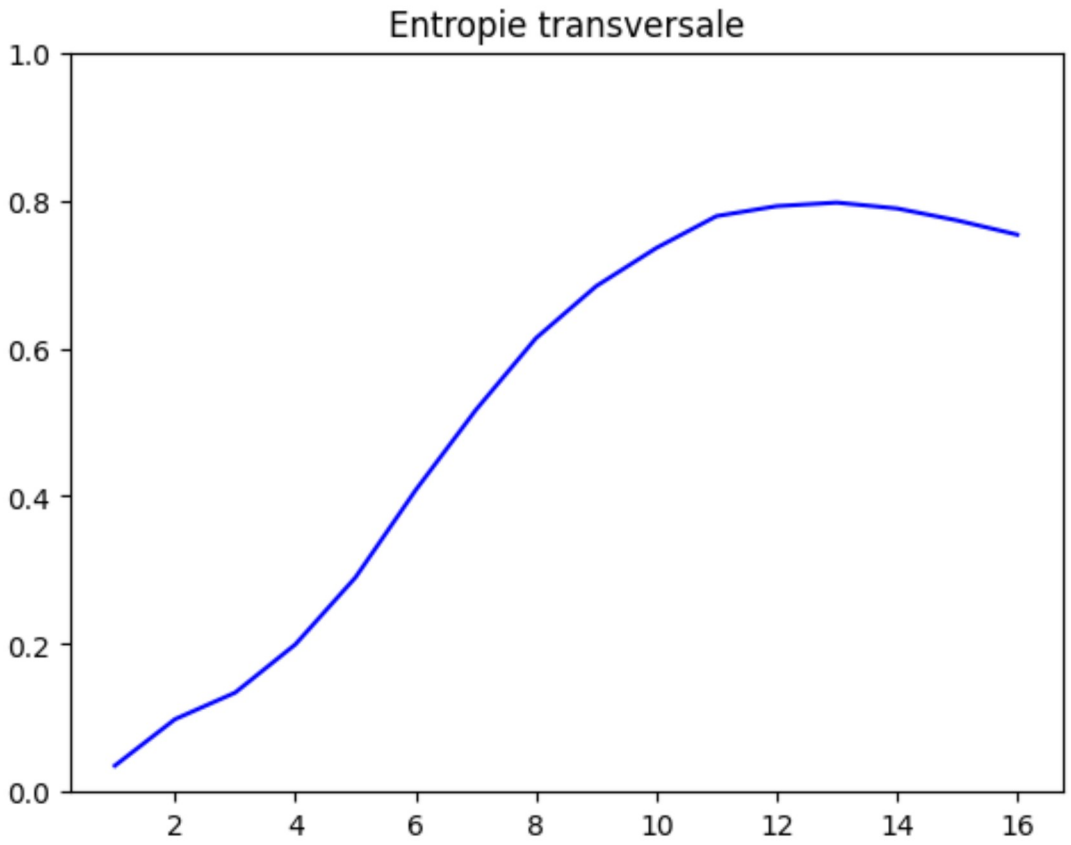
L'entropie transversale n'est pas proposée non plus mais peut être obtenue avec

pandas et son évolution peut être visualisée avec matplotlib .

In [25]:
```
# Calcul de l'entropie à chaque pas de temps avec la fonction entropy de scipy v
## Calcul de l'entropie
transversal_entropy = state_probs.apply(lambda row: entropy(row, base=2), axis=1
transversal_entropy
## Normalisation
k = len(biofam_expanded['states'].unique())
max_entropy = np.log2(k)
transversal_entropy_norm = transversal_entropy / max_entropy
transversal_entropy_norm
```

Out[25]:
```
timestep
1     0.035425
2     0.098278
3     0.134251
4     0.199422
5     0.290090
6     0.408142
7     0.516498
8     0.613623
9     0.683883
10    0.735539
11    0.778642
12    0.792120
13    0.796946
14    0.788995
15    0.772860
16    0.753571
dtype: float64
```

In [26]:
```
# Visualisation avec matplotlib
x = range(1,17)
fig, ax = plt.subplots()
ax.set_ylim(0,1)
ax.plot(x,transversal_entropy_norm,color="blue")
plt.title("Entropie transversale")
plt.show()
```



Séquence représentative

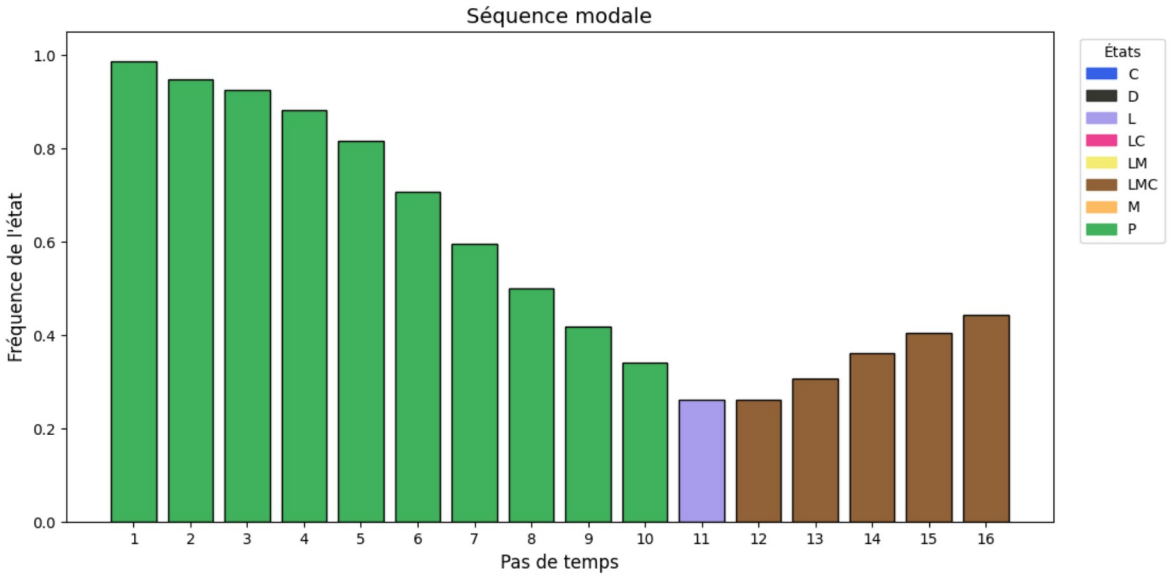
Pareillement pour la séquence représentative, il n'est pas possible de l'extraire directement avec `TanaT` mais il est possible de la construire avec `pandas` et de la visualiser avec `matplotlib` .

In [27]:
```
# Calcul de la séquence représentative avec pandas
modal_sequence = biofam_expanded.groupby('timestep')['states'].agg(lambda x: x.m
print("Séquence modale :")
print(modal_sequence.tolist())
```

```
Séquence modale :
['P', 'P', 'P', 'P', 'P', 'P', 'P', 'P', 'P', 'P', 'L', 'LMC', 'LMC', 'LMC', 'LMC
', 'LMC']
```

In [28]:
```
# Visualisation avec matplotlib
modal_states = state_probs.idxmax(axis=1)
modal_probs = state_probs.max(axis=1)

# Couleurs pour la légende
unique_states = state_probs.columns

fig, ax = plt.subplots(figsize=(12, 6))

for timestep in state_probs.index:
    state = modal_states[timestep]
    prob = modal_probs[timestep]
    ax.bar(timestep, prob, color=biofam_colors[state], edgecolor='black', width=


ax.set_xlabel('Pas de temps', fontsize=12)
ax.set_ylabel("Fréquence de l'état", fontsize=12)
```

```
ax.set_title('Séquence modale', fontsize=14)

ax.set_ylim(0, 1.05)
ax.set_xticks(state_probs.index)

# Création de la légende
legend_patches = [mpatches.Patch(color=biofam_colors[state], label=state) for st
ax.legend(handles=legend_patches, title='États', bbox_to_anchor=(1.02, 1), loc='

plt.show()
```



Descripteurs individuels

`TanaT` propose quelques descripteurs statistiques par le biais de la méthode `statistics` à appliquer sous sur l'objet contenant l'ensemble des séquences (`XSequencePool`) ou sur une seule séquence. Elle permet d'obtenir :

- la longueur minimale, maximale et moyenne au sein des séquences
- le nombre de séquences
- la taille de l'alphabet
- la longueur de la séquence
- l'entropie de la séquence
- le nombre d'états distincts au sein de la séquence
- le nombre de transitions
- la durée moyenne et médiane d'un état avec l'écart-type

In [29]:
```
# Ensemble des séquences
biofam_pool.statistics
```

Out[29]:
```
{'total_sequences': 2000,
 'avg_length': 16.0,
 'min_length': 16,
 'max_length': 16,
 'vocab_size': 8}
```

In [30]:
```
# Une seule séquence
biofam_pool[1].statistics
```

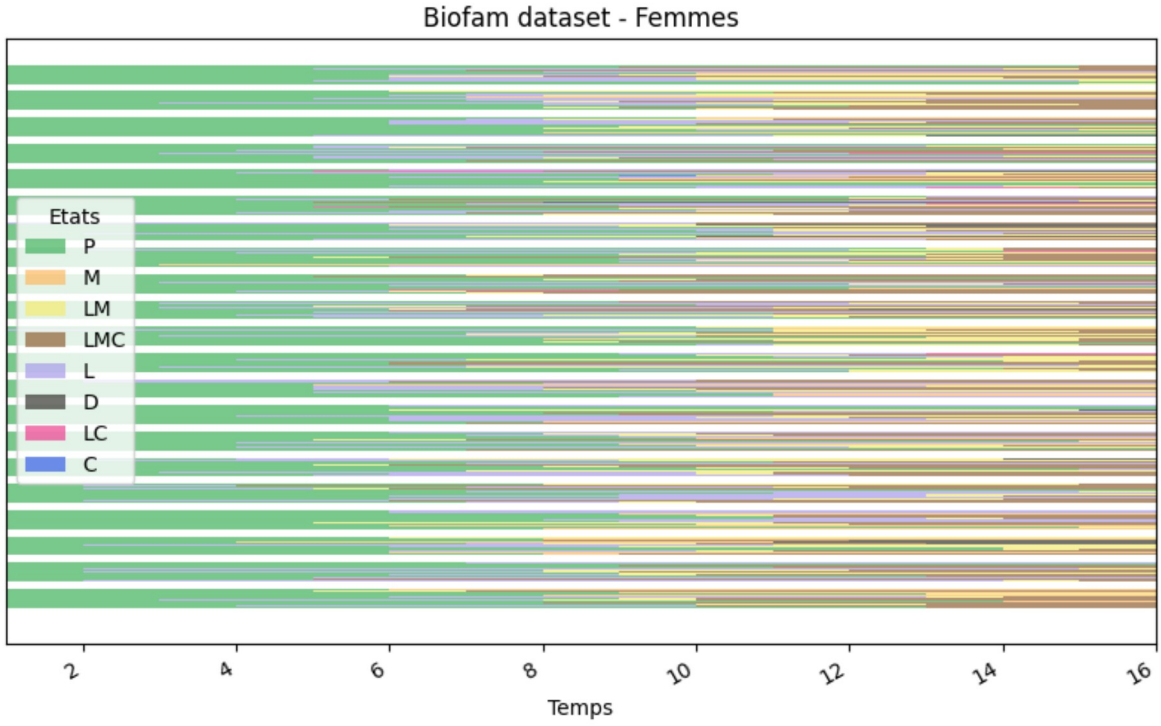
Out[30]:
```
{'length': 16.0,
 'entropy': 0.3372900666170139,
 'vocabulary_size': 2.0,
 'temporal_span': 15.0,
 'n_transitions': 1.0,
 'mean_duration': 0.9375,
 'median_duration': 1.0,
 'duration_std': 0.25,
 'vocab_size': 2}
```

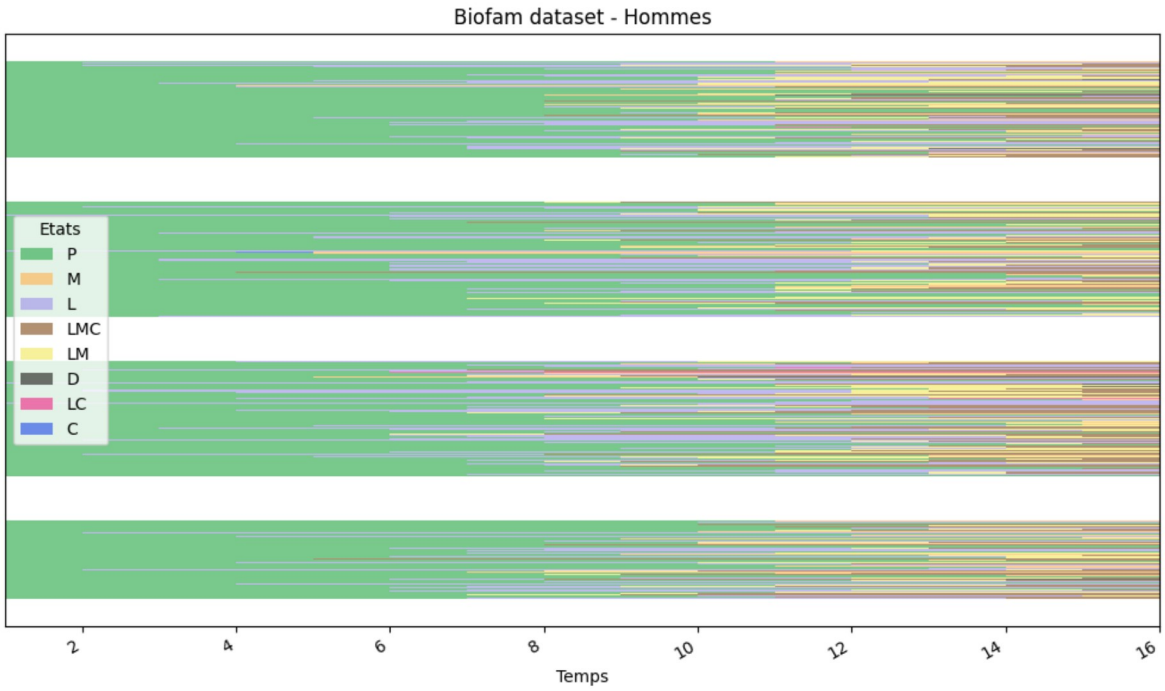
Il serait possible de calculer la complexité, la turbulence et le nombre de sous-séquences pour chaque séquence avec `pandas` mais pas directement avec `TanaT`.

Autres possibilités de visualisation

Les foncitons de visualisations de TanaT ne permettent pas d'obtenir directement plusieurs visuels selon un groupement mais il est tout à fait possible de générer deux visuels à partir de deux groupements filtré.

In [31]:
```
# Séparation de biofam en 2 selon le sexe (colonne sex)
criterion_man = StaticCriterion(query="sex == 'man'")
criterion_woman = StaticCriterion(query="sex == 'woman'")
man_seqpool = biofam_pool.filter(criterion_man)
woman_seqpool = biofam_pool.filter(criterion_woman)

# Visualisation
SequenceVisualizer.timeline(stacking_mode="flat") \
    .title("Biofam dataset - Femmes") \
    .colors(biofam_colors) \
    .marker(spacing=0.9) \
    .legend(show=True, title="Etats", loc="center left") \
    .x_axis(label="Temps") \
    .draw(woman_seqpool)

SequenceVisualizer.timeline(stacking_mode="flat") \
    .title("Biofam dataset - Hommes") \
    .colors(biofam_colors) \
    .marker(spacing=0.9) \
    .legend(show=True, title="Etats", loc="center left") \
    .x_axis(label="Temps") \
    .draw(man_seqpool)
```





Out[31]:

Il est également possible de préciser la palette de couleurs à utiliser avec la méthode `.colors()` et de gérer d'autres paramètres pour personnaliser les graphiques.

7. Obtention de typologies par l'analyse de séquences

L'une des application majeur de l'analyse de séquences est la construction de typologies : le regroupement des séquences partageant des propriétés similaires. L'identification de ces profils types permet ensuite de travailler avec des covariables afin d'expliquer la formation de ces différents groupes.

La construction de ces typologies se fait en deux étapes : l'évaluation des écarts entre chaque paire de séquence qui fournit une matrice de (di)similarités, puis l'application d'un algorithme de classification (clustering) sur cette matrice afin de former des groupes d'individus similaires (ou peu dissimilaires).

Pour chacune de ces étapes, le package `TanaT` propose plusieurs possibilité que nous allons illustrer sur le jeu de données `biofam` .

7.1 Distance entre les entités

Certaines métriques de distances reposent sur la distance entres les différents états. Pour cela, la librairie `TanaT` propose la distance de Hamming pour définir la distance entre deux états ou propose de coder sa propre métrique.

In [32]:
```
# Exemple de matrice des coût où toutes les différences sont fixées à 1 sauf pou
settings = HammingEntityMetricSettings()
hamming_metric = HammingEntityMetric(settings=settings)
hamming_metric.update_settings(
    cost={
        ("P", "L"): 0,
        ("L", "LC"): 2,
        ("M","LMC"): 2
    },
    default_value=1,  # Fallback cost for any unspecified comparison
)
```

In [33]:
```
seq1 = full_sequences[1]
seq2 = full_sequences[4]

print(seq1)
print(seq2)
```

```
P-P-P-P-P-P-P-P-P-P-P-P-P-P-M
P-P-P-P-P-P-L-L-L-L-L-L-L-LMC-LMC-LMC
```

In [34]:
```
criterion1 = QueryCriterion(query="id == 1")
criterion2 = QueryCriterion(query="id == 4")
biofam_seq1 = biofam_pool.filter(criterion1, level="sequence")
biofam_seq2 = biofam_pool.filter(criterion2, level="sequence")
hamming_metric(biofam_seq1[1][15], biofam_seq2[4][15]) # Comparaison du 16eme (d
```

Out[34]: 2

7.2 Distance entre les séquences

La librairie TanaT propose de nombreuses métrique pour calculer la dissimilarité entre les séquences :

- **Distances d'édition** : Dynamic Time Warping, Soft Dynamic Time Wrapping, Edit distance (OM), Linear Pairwise Sequence Metric ($\rightarrow$ nécessite une métrique entre entités).
- **Mesures basées sur le nombre d'attributs communs** : LCP, LCS.
- **Distances entre les distributions d'états** : $\backslash \mathrm{Chi}^2$.

La librairie propose également la possibilité d'implémenter sa propre métrique entre séquences.

In [35]:
```
settings = EditSequenceMetricSettings()
edit_metric = EditSequenceMetric(settings=settings)

# Calcul de la distance d'édition entre deux séquences. Coût = 1 entre tous les
edit_metric(biofam_seq1[1], biofam_seq2[4])
```

Out[35]: 10.0

In [36]:
```
start_time = time.time()
# Calcul de la matrice de dissimilarité pour l'ensemble des séquences
dm = edit_metric.compute_matrix(biofam_pool)
dm

time_taken = time.time() - start_time
print(str(time_taken)+" s")
```

```
┌─ EditSequenceMetric
│
│ Progress: 100%|████████| 1999000/1999000 [00:01<00:00, 1601549.38it/s]
│
└─ Done (2000x2000 matrix, 1.81s)
1.8142404556274414 s
```

7.3 Clustering

Une fois la matrice de dissimilarité calculée, la seconde étape consiste à regrouper les trajectoires similaires au moyen d'un algorithme de clustering. TanaT propose directement plusieurs algorithmes de clustering et inclut directement le résultat du clustering à l'objet regroupant les séquences.

Les 3 algorithmes inclus direcemtent sont :

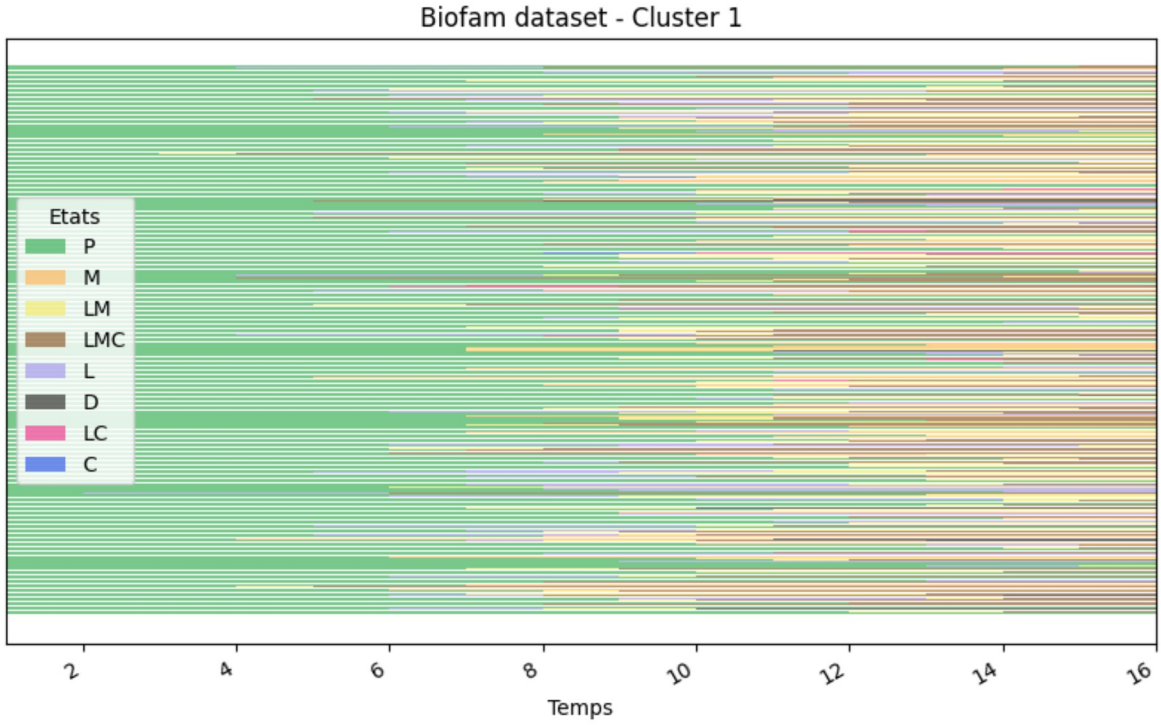
- hierarchical clustering (agglomerative nesting, AGNES)
- Clustering Large Application (CLARA)
- Partitioning Around Medoids (PAM)

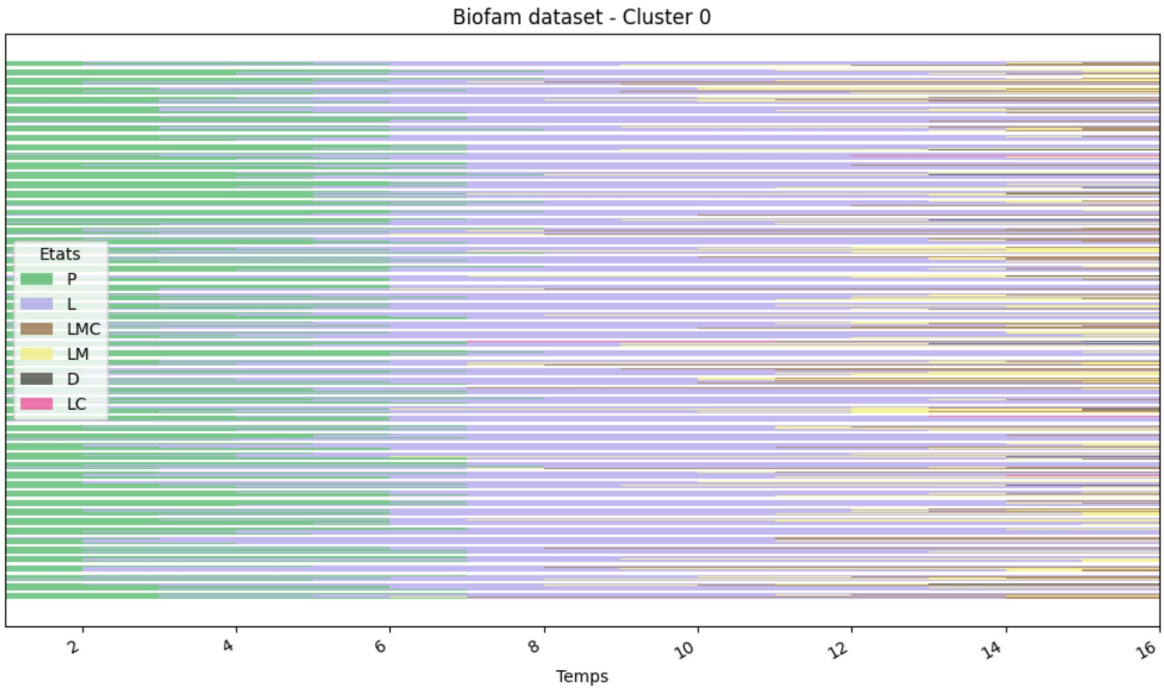
A partir de la matrice, il est également possible de faire appel à d'autres librairies pour avoir accès à d'autres algorithmes (notamment Fuzzy Clustering (FANNY)).

In [37]:
```
start_time = time.time()
hc_settings = HierarchicalClustererSettings(
    metric=edit_metric,  # metrique au niveau des séquences
    cluster_column="hclass",  # nom de la colonne pour le clustering
)
clusterer = HierarchicalClusterer(settings=hc_settings)

clusterer.fit(biofam_pool)

time_taken = time.time() - start_time
print(str(time_taken)+" s")
```

```
┌─ HierarchicalClusterer
│
│ Step 1/3: Computing distance matrix
│
│   ┌─ EditSequenceMetric
│   │
│   │ Using cached matrix (instance cache hit)
│   │
│   └─ Done (2000x2000 matrix, 0.00s)
│
│ Step 2/3: Performing hierarchical clustering
│
│ Step 3/3: Updating static data
│
└─ Done (0.13s)
0.12659287452697754 s
```

In [38]:
```python
# Séparation de biofam en 2 selon le cluster
criterion1 = StaticCriterion(query="hclass == 1")
criterion0 = StaticCriterion(query="hclass == 0")
seqpool1 = biofam_pool.filter(criterion1)
seqpool0 = biofam_pool.filter(criterion0)

# Visualisation
SequenceVisualizer.timeline(stacking_mode="flat") \
    .title("Biofam dataset - Cluster 1") \
    .colors(biofam_colors) \
    .marker(spacing=0.9) \
    .legend(show=True, title="Etats", loc="center left") \
    .x_axis(label="Temps") \
    .draw(seqpool1)

SequenceVisualizer.timeline(stacking_mode="flat") \
    .title("Biofam dataset - Cluster 0") \
    .colors(biofam_colors) \
    .marker(spacing=0.9) \
    .legend(show=True, title="Etats", loc="center left") \
    .x_axis(label="Temps") \
    .draw(seqpool0)
```





Out[38]:

Nous obtenons bien alors deux clusters. Le cluster 1 est composé d'individus qui restent assez longtemps chez leurs parents et quittent l'état "P" pour se marier ou avoir un enfant ("LMC", "LM", "LC") et le cluster 0 est composé d'individus qui quittent le domicil parental pour vivre seuls (état "L") avant de se marier ou avoir des enfants.

7.4 Cas du multichannel

L'analyse de séquences ne se limite pas qu'au cas unidimensionnel comme illustré précedemment et peut-être également intéressantes dans un contexte multidiomensionnel pour plusieurs individus reliés entre eux ou plusieurs aspects d'un même individu.

Le processus analytique dans le cadre multidimensionnel reste globalement le même : il s'agit toujours de définir des séquences à partir d'un certain alphabet d'états, de choisir le coût du passage d'un état à un autre, de calculer la dissimilarité entre les trajectoires et d'appliquer une algorithme de clustering. L'enjeu devient alors de choisir le moment auquel considérer l'aspect multidimensionnel. Quatre approches se distingues : extension de l'alphabet (EA, *Extended Alphabet*), association des matrices de coût (CAT, *Cost Additive Trick*), association des matrices de dissimilarité (DAT, *Distance Additive Trick*) et combinaison des clusters (CombT, *Combining Trajectories*) qui sont toutes implémentables en Python avec `TanaT`.

Nous allons illustrer ces possibilités avec le jeu de données `fb_careers`.

Extended Alphabet

On crée un nouvel état représentant la combinaison des états auxquels on s'intéresse. Puis on applique le même processus que présenté précedemment.

In [39]:
```
start_time = time.time()
# Création d'un nouvel alphabet en fusinant deux colonnes
simple_fb["extended_alphabet"] = simple_fb[colname1].astype(str) + "_" + simple_

ea_settings = {
    "id_column": "player_id",
    "start_column": "begin",  # On utilise la nouvelle colonne dépliée
    "default_end_value":2023,
    "entity_features": ["extended_alphabet"],
}
ea_pool = StateSequencePool(simple_fb, ea_settings)

hc_settings = HierarchicalClustererSettings(
    metric="edit",  # metrique au niveau des séquences
    cluster_column="hclass_ea",  # nom de la colonne pour le clustering
)
clusterer = HierarchicalClusterer(settings=hc_settings)

clusterer.fit(ea_pool)

time_taken = time.time() - start_time
print(str(time_taken)+" s")
```

```
┌─ HierarchicalClusterer
│
│ Step 1/3: Computing distance matrix
│
│   ┌─ EditSequenceMetric
│   │
│   │ Progress: 100%|██████████| 124750/124750 [00:00<00:00, 2798775.22it/s]
```

```
│   │
│   └─ Done (500x500 matrix, 0.26s)
│
│ Step 2/3: Performing hierarchical clustering
│
│ Step 3/3: Updating static data
│
└─ Done (0.27s)
0.2749323844909668 s
```

Il est important de note qu'avec cette définition, le coût pour passer d'un état à un autre est toujours le même, quand bien même les deux états composites contiennent un même sous-état.

Exemple :

- Coût((Etat1 + Etat A) , (Etat1 + Etat B)) = 1
- Coût((Etat1 + Etat A) , (Etat2 + Etat B)) = 1

Cost Additive Trick

On additionne les matrices de coûts entre les domaines afin de mieux prendre en compte la similarité de sous-états.

Exemple :

- Coût((Etat 1 + Etat A) , (Etat 1 + Etat B)) = Coût(Etat 1, Etat 1) + Coût(Etat A, Etat B) = 1
- Coût((Etat 1 + Etat A) , (Etat 2 + Etat B)) = Coût(Etat 1, Etat 2) + Coût(Etat A, Etat B) = 2

In [40]:
```
start_time = time.time()
# Métrique entre entités
hamming_metric.update_settings(
    cost={
        ("0_0", "0_0"): 0,
        ("0_0", "0_1"): 1,
        ("0_0", "0_2"): 1,
        ("0_0", "1_0"): 1,
        ("0_0", "2_0"): 1,

        ("0_1", "0_1"): 0,
        ("0_1", "0_2"): 1,
        ("0_1", "1_1"): 1,
        ("0_1", "2_1"): 1,

        ("0_2", "0_2"): 0,
        ("0_2", "1_2"): 1,
        ("0_2", "2_2"): 1,

        ("1_0", "1_0"): 0,
        ("1_0", "1_1"): 1,
        ("1_0", "1_2"): 1,
        ("1_0", "2_0"): 1,
```

```
        ("1_1", "1_1"): 0,
        ("1_1", "1_2"): 1,

        ("1_2", "1_2"): 0,
        ("1_2", "2_2"): 1,

        ("2_0", "2_0"): 0,
        ("2_0", "2_1"): 1,
        ("2_0", "2_2"): 1,

        ("2_1", "2_1"): 0,
        ("2_1", "2_2"): 1,

        ("2_2", "2_2"): 0,
    },
    default_value=2,  # Fallback cost for any unspecified comparison
)

settings = EditSequenceMetricSettings(entity_metric=hamming_metric)
edit_metric = EditSequenceMetric(settings=settings)


hc_settings = HierarchicalClustererSettings(
    metric= edit_metric,  # metrique au niveau des séquences
    cluster_column="hclass_cat",  # nom de la colonne pour le clustering

)
clusterer = HierarchicalClusterer(settings=hc_settings)

clusterer.fit(ea_pool)


time_taken = time.time() - start_time
print(str(time_taken)+" s")
```

```
┌─ HierarchicalClusterer
│
│ Step 1/3: Computing distance matrix
│
│   ┌─ EditSequenceMetric
│   │
│   │ Progress: 100%|██████████| 124750/124750 [00:00<00:00, 146897.85it/s]
│   │
│   └─ Done (500x500 matrix, 1.07s)
│
│ Step 2/3: Performing hierarchical clustering
│
│ Step 3/3: Updating static data
│
└─ Done (1.08s)
1.0824053287506104 s
```

Distance Additive Trick

Le package TanaT permet de constuire des objets trajectoires qui regroupent plusieurs séquences (qui peuvent être de natures différentes (évènements, intervalles et états)) et permettent de réaliser directement l'analyse sur plusieurs cannaux avec l'approche DAT (Distance Additive Trick) qui consiste à sommer les matrice de dissimilarité (distance).

In [41]:
```
start_time = time.time()
# Définition des deux séquences
seq1_settings = {
    "id_column": "player_id",
    "start_column": "begin",  # On utilise la nouvelle colonne dépliée
    "default_end_value":2023,
    "entity_features": [colname1],
}
seq1_pool = StateSequencePool(simple_fb, seq1_settings)

seq2_settings = {
    "id_column": "player_id",
    "start_column": "begin",  # On utilise la nouvelle colonne dépliée
    "default_end_value":2023,
    "entity_features": [colname2],
}
seq2_pool = StateSequencePool(simple_fb, seq2_settings)


# Trajectoire
trajectory_pool = TrajectoryPool.init_empty()
trajectory_pool.add_sequence_pool(seq1_pool, "states1")
trajectory_pool.add_sequence_pool(seq2_pool, "states2")
trajectory_pool.update_settings(intersection=False)

# Metrique aggrégative
settings = AggregationTrajectoryMetricSettings()
mean_agg_metric = AggregationTrajectoryMetric(settings=settings)

# Clustering avec nouvelle métrique
hc_settings = HierarchicalClustererSettings(
    metric=mean_agg_metric,  # metrique au niveau des séquences
    cluster_column="hclass_dat",  # nom de la colonne pour le clustering

)
clusterer = HierarchicalClusterer(settings=hc_settings)
clusterer.fit(trajectory_pool)


time_taken = time.time() - start_time
print(str(time_taken)+" s")
```

```
┌─ HierarchicalClusterer
│
│ Step 1/3: Computing distance matrix
│
│   ┌─ AggregationTrajectoryMetric
│   │ Aggregation metric requires intersection mode. Enforcing intersection on tr
ajectory pool copy.
│
│   │
│   │ Step 1/2: Computing sequence metrics
│   │
│   │   (1/2) states1 → LinearPairwiseSequenceMetric
│   │
│   │ Progress: 100%|██████████| 124750/124750 [00:00<00:00, 136521.93it/s]
│   │
│   │   (2/2) states2 → LinearPairwiseSequenceMetric
│   │
│   │ Progress: 100%|██████████| 124750/124750 [00:00<00:00, 540705.29it/s]
│   │
│   │ Step 2/2: Aggregating (mean)
│   │
│   │ Progress: 100%|██████████| 124750/124750 [00:01<00:00, 112223.25it/s]
│   │
│   └─ Done (500x500 matrix, 3.40s)
│
│ Step 2/3: Performing hierarchical clustering
│
│ Step 3/3: Updating static data
│
└─ Done (3.41s)
3.414275884628296 s
```

Combining Typologies

L'approche CombT consiste à réaliser le clustering sur chaque domaine de manière indépendante puis à créer des clusters croisant les différentes possibilités de cluster. Dans le cas où les deux clustering ammènent à deux clusters chacun, nous obtenons finalement 4 clusters finaux.

In [42]:
```python
start_time = time.time()
# Clustering séquence domaine 1
hc_settings1 = HierarchicalClustererSettings(
    metric="edit",  # metrique au niveau des séquences
    cluster_column="hclass_seq1",  # nom de la colonne pour le clustering
)
clusterer1 = HierarchicalClusterer(settings=hc_settings1)
clusterer1.fit(seq1_pool)
clusterer1

# Clustering séquence domaine 2
hc_settings2 = HierarchicalClustererSettings(
    metric="edit",  # metrique au niveau des séquences
    cluster_column="hclass_seq2",  # nom de la colonne pour le clustering
)
clusterer2 = HierarchicalClusterer(settings=hc_settings2)
clusterer2.fit(seq2_pool)
clusterer2
```

```
# Croisement des clusters
combT_cluster = seq1_pool.get_static_feature("hclass_seq1").astype(str) + "_" +


time_taken = time.time() - start_time
print(str(time_taken)+" s")
```

```
┌─ HierarchicalClusterer
│
│ Step 1/3: Computing distance matrix
│
│    ┌─ EditSequenceMetric
│    │
│    │ Progress: 100%|██████████| 124750/124750 [00:00<00:00, 2649635.52it/s]
│    │
│    └─ Done (500x500 matrix, 0.21s)
│
│ Step 2/3: Performing hierarchical clustering
│
│ Step 3/3: Updating static data
│
└─ Done (0.21s)
┌─ HierarchicalClusterer
│
│ Step 1/3: Computing distance matrix
│
│    ┌─ EditSequenceMetric
│    │
│    │ Progress: 100%|██████████| 124750/124750 [00:00<00:00, 3757311.37it/s]
│    │
│    └─ Done (500x500 matrix, 0.19s)
│
│ Step 2/3: Performing hierarchical clustering
│
│ Step 3/3: Updating static data
│
└─ Done (0.19s)
0.41217994689941406 s
```

8. Analyse de survie

La librairie `TanaT` inclut plusieurs objets permetant de réaliser des analyses de survie : `SurvivalAnalysis("coxnet")` pour construire un modèle de Cox (avec sélection de variable via Elastic-Net) et `SurvivalAnalysis("tree")` pour construire un modèle d'arbre de survie.

Nous allons faire un exemple avec l'analyse de la survenue de l'évènement mariage (LMC, LM et M) selon les covariables sexe et date de naissance. Pour cet exemple, nous utilisons le jeu de données biofam.

In [43]:
```
biofam = pd.read_csv("../biofam.csv").dropna(axis=0,subset="id")
biofam_static = pd.read_csv("../biofam_sts.csv",dtype={"birthyr":"Int64"})

# Timestep pour lisible par Tanat
```

```
biofam_expanded = biofam.copy()
biofam_expanded['timestep'] = biofam_expanded.apply(
    lambda row: list(range(int(row['begin']), int(row['end']) + 1)), axis=1
)
biofam_expanded = biofam_expanded.explode('timestep')
biofam_expanded["timestep"] = biofam_expanded["timestep"].astype("Int64")
padded_years = biofam_expanded["timestep"].astype(str).str.zfill(4)
biofam_expanded["timestep"] = pd.to_datetime(padded_years, format='%Y') # Format

# Transformation de birthyear en catégorielle avec 4 niveaux
biofam_static["birthyr_cat"] = pd.cut(biofam_static["birthyr"],bins=4,labels=ran
```

In [44]:
```
# Définition de l'objet biofam_pool
biofam_settings = {
    "id_column": "id",
    "start_column": "timestep",
    "default_end_value": pd.to_datetime("2002", format="%Y"),
    "entity_features": ["states"],
}
biofam_pool = StateSequencePool(biofam_expanded, biofam_settings)

# Ajout d'information statiques : sexe (sex)
static_df = biofam_static[["id", "sex", "birthyr_cat"]].copy()
biofam_pool = biofam_pool.add_static_features(static_df, override=True)
```

Modèle Coxnet

In [45]:
```
# Création objet coxnet
surv = SurvivalAnalysis("coxnet")
biofam_pool.zero_from_position(0)

# Séparation train / test
all_ids = list(biofam_pool.unique_ids)
train_ids = set(random.sample(all_ids, int(0.99 * len(all_ids))))
test_ids = set(all_ids) - train_ids

train_pool = biofam_pool.subset(train_ids)
test_pool = biofam_pool.subset(test_ids)

# Extraction de la durée de survie (temps avant d'être marié)
fit_results = surv.fit(sequence_pool=train_pool, query="states == 'LM' | states

# Prédiction
survival_predictions = surv.predict_survival_function(sequence_or_pool=test_pool

# Visualisation
plt.figure(figsize=(12, 8))
for i, sf in enumerate(survival_predictions):
    plt.step(sf.x, sf.y, where="post", label=f"Patient {i+1}")

plt.title("Fonctions de survie prédites")
plt.xlabel("Temps (jours)")
plt.ylabel("Probabilité de ne pas être marié (LM,LMC,M)")
plt.grid(True)
plt.legend(loc="best")
plt.show()
```

```
No sequence found with id 10
No sequence found with id 38
No sequence found with id 40
No sequence found with id 42
No sequence found with id 45
No sequence found with id 46
No sequence found with id 49
No sequence found with id 55
No sequence found with id 60
No sequence found with id 74
No sequence found with id 77
No sequence found with id 78
No sequence found with id 81
No sequence found with id 90
No sequence found with id 99
No sequence found with id 118
No sequence found with id 124
No sequence found with id 134
No sequence found with id 153
No sequence found with id 156
No sequence found with id 164
No sequence found with id 165
No sequence found with id 187
No sequence found with id 208
No sequence found with id 209
No sequence found with id 212
No sequence found with id 213
No sequence found with id 221
No sequence found with id 222
No sequence found with id 226
No sequence found with id 228
No sequence found with id 229
No sequence found with id 230
No sequence found with id 238
No sequence found with id 241
No sequence found with id 249
No sequence found with id 257
No sequence found with id 270
No sequence found with id 272
No sequence found with id 273
No sequence found with id 274
No sequence found with id 275
No sequence found with id 278
No sequence found with id 291
No sequence found with id 300
No sequence found with id 302
No sequence found with id 304
No sequence found with id 307
No sequence found with id 319
No sequence found with id 321
No sequence found with id 361
No sequence found with id 367
No sequence found with id 369
No sequence found with id 373
No sequence found with id 377
No sequence found with id 378
No sequence found with id 383
No sequence found with id 386
No sequence found with id 390
No sequence found with id 402
```

```
No sequence found with id 404
No sequence found with id 407
No sequence found with id 419
No sequence found with id 421
No sequence found with id 422
No sequence found with id 423
No sequence found with id 424
No sequence found with id 427
No sequence found with id 432
No sequence found with id 439
No sequence found with id 440
No sequence found with id 443
No sequence found with id 445
No sequence found with id 446
No sequence found with id 447
No sequence found with id 449
No sequence found with id 454
No sequence found with id 456
No sequence found with id 463
No sequence found with id 481
No sequence found with id 482
No sequence found with id 484
No sequence found with id 488
No sequence found with id 492
No sequence found with id 493
No sequence found with id 494
No sequence found with id 496
No sequence found with id 498
No sequence found with id 507
No sequence found with id 508
No sequence found with id 510
No sequence found with id 516
No sequence found with id 517
No sequence found with id 518
No sequence found with id 519
No sequence found with id 521
No sequence found with id 523
No sequence found with id 531
No sequence found with id 533
No sequence found with id 542
No sequence found with id 543
No sequence found with id 558
No sequence found with id 566
No sequence found with id 567
No sequence found with id 568
No sequence found with id 570
No sequence found with id 571
No sequence found with id 572
No sequence found with id 577
No sequence found with id 585
No sequence found with id 586
No sequence found with id 588
No sequence found with id 589
No sequence found with id 596
No sequence found with id 600
No sequence found with id 606
No sequence found with id 608
No sequence found with id 622
No sequence found with id 624
No sequence found with id 658
```

```
No sequence found with id 668
No sequence found with id 675
No sequence found with id 692
No sequence found with id 706
No sequence found with id 707
No sequence found with id 715
No sequence found with id 717
No sequence found with id 737
No sequence found with id 738
No sequence found with id 739
No sequence found with id 740
No sequence found with id 748
No sequence found with id 753
No sequence found with id 758
No sequence found with id 766
No sequence found with id 767
No sequence found with id 769
No sequence found with id 773
No sequence found with id 774
No sequence found with id 775
No sequence found with id 779
No sequence found with id 781
No sequence found with id 787
No sequence found with id 789
No sequence found with id 797
No sequence found with id 815
No sequence found with id 820
No sequence found with id 821
No sequence found with id 824
No sequence found with id 826
No sequence found with id 829
No sequence found with id 834
No sequence found with id 838
No sequence found with id 843
No sequence found with id 844
No sequence found with id 853
No sequence found with id 855
No sequence found with id 868
No sequence found with id 872
No sequence found with id 879
No sequence found with id 881
No sequence found with id 887
No sequence found with id 904
No sequence found with id 909
No sequence found with id 910
No sequence found with id 911
No sequence found with id 914
No sequence found with id 919
No sequence found with id 926
No sequence found with id 927
No sequence found with id 931
No sequence found with id 933
No sequence found with id 936
No sequence found with id 960
No sequence found with id 962
No sequence found with id 971
No sequence found with id 972
No sequence found with id 973
No sequence found with id 980
No sequence found with id 984
```

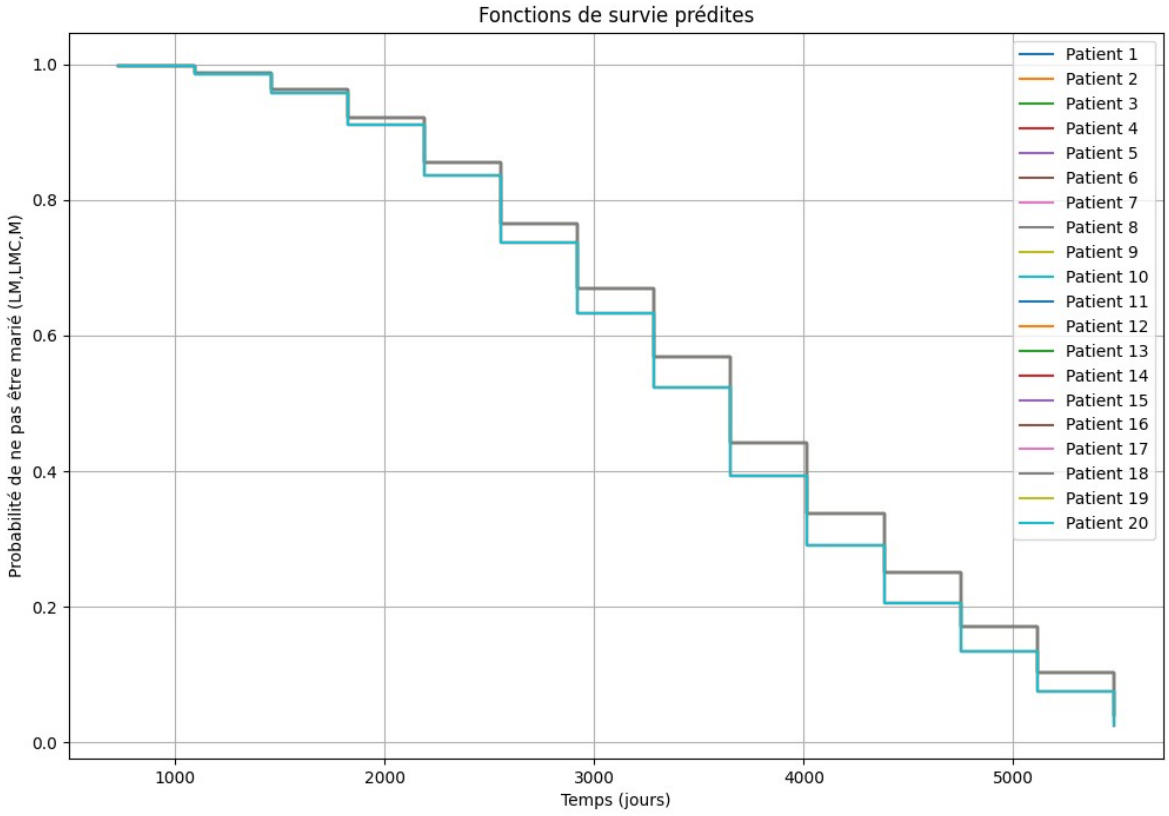
```
No sequence found with id 985
No sequence found with id 986
No sequence found with id 991
No sequence found with id 993
No sequence found with id 1017
No sequence found with id 1021
No sequence found with id 1023
No sequence found with id 1024
No sequence found with id 1028
No sequence found with id 1029
No sequence found with id 1031
No sequence found with id 1034
No sequence found with id 1037
No sequence found with id 1040
No sequence found with id 1041
No sequence found with id 1047
No sequence found with id 1064
No sequence found with id 1084
No sequence found with id 1087
No sequence found with id 1094
No sequence found with id 1095
No sequence found with id 1096
No sequence found with id 1113
No sequence found with id 1119
No sequence found with id 1121
No sequence found with id 1123
No sequence found with id 1130
No sequence found with id 1134
No sequence found with id 1139
No sequence found with id 1142
No sequence found with id 1148
No sequence found with id 1151
No sequence found with id 1158
No sequence found with id 1179
No sequence found with id 1197
No sequence found with id 1200
No sequence found with id 1201
No sequence found with id 1207
No sequence found with id 1210
No sequence found with id 1215
No sequence found with id 1225
No sequence found with id 1229
No sequence found with id 1236
No sequence found with id 1237
No sequence found with id 1246
No sequence found with id 1253
No sequence found with id 1259
No sequence found with id 1263
No sequence found with id 1265
No sequence found with id 1266
No sequence found with id 1268
No sequence found with id 1276
No sequence found with id 1290
No sequence found with id 1305
No sequence found with id 1308
No sequence found with id 1317
No sequence found with id 1332
No sequence found with id 1333
No sequence found with id 1346
No sequence found with id 1353
```

```
No sequence found with id 1355
No sequence found with id 1365
No sequence found with id 1368
No sequence found with id 1380
No sequence found with id 1383
No sequence found with id 1387
No sequence found with id 1390
No sequence found with id 1398
No sequence found with id 1411
No sequence found with id 1412
No sequence found with id 1414
No sequence found with id 1415
No sequence found with id 1437
No sequence found with id 1444
No sequence found with id 1450
No sequence found with id 1454
No sequence found with id 1456
No sequence found with id 1457
No sequence found with id 1459
No sequence found with id 1460
No sequence found with id 1462
No sequence found with id 1467
No sequence found with id 1469
No sequence found with id 1473
No sequence found with id 1498
No sequence found with id 1501
No sequence found with id 1504
No sequence found with id 1514
No sequence found with id 1515
No sequence found with id 1516
No sequence found with id 1519
No sequence found with id 1527
No sequence found with id 1533
No sequence found with id 1537
No sequence found with id 1541
No sequence found with id 1542
No sequence found with id 1550
No sequence found with id 1561
No sequence found with id 1562
No sequence found with id 1565
No sequence found with id 1576
No sequence found with id 1588
No sequence found with id 1591
No sequence found with id 1592
No sequence found with id 1604
No sequence found with id 1608
No sequence found with id 1610
No sequence found with id 1611
No sequence found with id 1619
No sequence found with id 1634
No sequence found with id 1635
No sequence found with id 1636
No sequence found with id 1638
No sequence found with id 1642
No sequence found with id 1660
No sequence found with id 1661
No sequence found with id 1662
No sequence found with id 1664
No sequence found with id 1684
No sequence found with id 1687
```

```
No sequence found with id 1701
No sequence found with id 1710
No sequence found with id 1717
No sequence found with id 1719
No sequence found with id 1723
No sequence found with id 1733
No sequence found with id 1735
No sequence found with id 1740
No sequence found with id 1748
No sequence found with id 1749
No sequence found with id 1759
No sequence found with id 1765
No sequence found with id 1766
No sequence found with id 1771
No sequence found with id 1784
No sequence found with id 1785
No sequence found with id 1792
No sequence found with id 1794
No sequence found with id 1804
No sequence found with id 1807
No sequence found with id 1817
No sequence found with id 1828
No sequence found with id 1832
No sequence found with id 1838
No sequence found with id 1843
No sequence found with id 1852
No sequence found with id 1855
No sequence found with id 1859
No sequence found with id 1867
No sequence found with id 1882
No sequence found with id 1883
No sequence found with id 1887
No sequence found with id 1888
No sequence found with id 1905
No sequence found with id 1906
No sequence found with id 1919
No sequence found with id 1929
No sequence found with id 1931
No sequence found with id 1932
No sequence found with id 1933
No sequence found with id 1938
No sequence found with id 1939
No sequence found with id 1944
No sequence found with id 1949
No sequence found with id 1951
No sequence found with id 1952
No sequence found with id 1976
No sequence found with id 1980
No sequence found with id 1982
No sequence found with id 2000
No sequence found with id 2006
No sequence found with id 2008
No sequence found with id 2009
No sequence found with id 2012
No sequence found with id 2016
No sequence found with id 2021
No sequence found with id 2022
No sequence found with id 2024
No sequence found with id 2025
No sequence found with id 2026
```

```
No sequence found with id 2034
No sequence found with id 2051
No sequence found with id 2060
No sequence found with id 2069
No sequence found with id 2077
No sequence found with id 2088
No sequence found with id 2089
No sequence found with id 2090
No sequence found with id 2093
No sequence found with id 2094
No sequence found with id 2105
No sequence found with id 2121
No sequence found with id 2124
No sequence found with id 2132
No sequence found with id 2133
No sequence found with id 2134
No sequence found with id 2135
No sequence found with id 2141
No sequence found with id 2143
No sequence found with id 2154
No sequence found with id 2166
No sequence found with id 2171
No sequence found with id 2172
No sequence found with id 2178
No sequence found with id 2181
No sequence found with id 2188
No sequence found with id 2199
No sequence found with id 2201
No sequence found with id 2203
No sequence found with id 2208
No sequence found with id 2211
No sequence found with id 2212
No sequence found with id 2221
No sequence found with id 2227
No sequence found with id 2231
No sequence found with id 2232
No sequence found with id 2233
No sequence found with id 2234
No sequence found with id 2240
No sequence found with id 2250
No sequence found with id 2252
No sequence found with id 2253
No sequence found with id 2263
No sequence found with id 2264
No sequence found with id 2278
No sequence found with id 2288
No sequence found with id 2293
No sequence found with id 2299
No sequence found with id 2300
No sequence found with id 2304
No sequence found with id 2309
No sequence found with id 2311
No sequence found with id 2317
No sequence found with id 2320
No sequence found with id 2321
No sequence found with id 2323
No sequence found with id 2324
No sequence found with id 2329
No sequence found with id 2353
No sequence found with id 2367
```

```
No sequence found with id 2368
No sequence found with id 2372
No sequence found with id 2376
No sequence found with id 2387
No sequence found with id 2391
No sequence found with id 2392
No sequence found with id 2395
No sequence found with id 2403
No sequence found with id 2410
No sequence found with id 2416
No sequence found with id 2423
No sequence found with id 2430
No sequence found with id 2431
No sequence found with id 2435
No sequence found with id 2439
No sequence found with id 2446
No sequence found with id 2462
No sequence found with id 2475
No sequence found with id 2480
No sequence found with id 2490
No sequence found with id 2494
No sequence found with id 2495
No sequence found with id 2500
No sequence found with id 2514
No sequence found with id 2515
No sequence found with id 2529
No sequence found with id 2543
No sequence found with id 2553
No sequence found with id 2564
No sequence found with id 2572
No sequence found with id 2579
No sequence found with id 2593
```



Tree Survival

In [46]:
```
surv = SurvivalAnalysis("tree")
biofam_pool.zero_from_position(0)

# Séparation train / test
all_ids = list(biofam_pool.unique_ids)
train_ids = set(random.sample(all_ids, int(0.99 * len(all_ids))))
test_ids = set(all_ids) - train_ids

train_pool = biofam_pool.subset(train_ids)
test_pool = biofam_pool.subset(test_ids)

surv_res = surv.get_survival_array(sequence_pool=train_pool, query="states == 'L
fit_results = surv.fit(sequence_pool=train_pool, query="states == 'LM' | states

survival_predictions = surv.predict_survival_function(sequence_or_pool=test_pool

# Visualisation
plt.figure(figsize=(12, 8))
for i, sf in enumerate(survival_predictions):
    plt.step(sf.x, sf.y, where="post", label=f"Patient {i+1}")

plt.title("Predicted Survival Functions")
plt.xlabel("Time (days)")
plt.ylabel("Survival Probability")
plt.grid(True)
plt.legend(loc="best")
plt.show()
```

```
No sequence found with id 10
No sequence found with id 38
No sequence found with id 40
No sequence found with id 42
No sequence found with id 45
No sequence found with id 46
No sequence found with id 49
No sequence found with id 55
No sequence found with id 60
No sequence found with id 74
No sequence found with id 77
No sequence found with id 78
No sequence found with id 81
No sequence found with id 90
No sequence found with id 99
No sequence found with id 118
No sequence found with id 124
No sequence found with id 134
No sequence found with id 153
No sequence found with id 156
No sequence found with id 164
No sequence found with id 165
No sequence found with id 187
No sequence found with id 208
No sequence found with id 209
No sequence found with id 212
No sequence found with id 213
No sequence found with id 221
No sequence found with id 222
No sequence found with id 226
No sequence found with id 228
No sequence found with id 229
No sequence found with id 230
No sequence found with id 238
No sequence found with id 239
No sequence found with id 241
No sequence found with id 249
No sequence found with id 257
No sequence found with id 270
No sequence found with id 272
No sequence found with id 273
No sequence found with id 274
No sequence found with id 275
No sequence found with id 278
No sequence found with id 291
No sequence found with id 300
No sequence found with id 302
No sequence found with id 304
No sequence found with id 307
No sequence found with id 319
No sequence found with id 321
No sequence found with id 361
No sequence found with id 367
No sequence found with id 369
No sequence found with id 373
No sequence found with id 377
No sequence found with id 378
No sequence found with id 383
No sequence found with id 386
No sequence found with id 390
```

```
No sequence found with id 402
No sequence found with id 404
No sequence found with id 407
No sequence found with id 419
No sequence found with id 421
No sequence found with id 422
No sequence found with id 423
No sequence found with id 424
No sequence found with id 427
No sequence found with id 432
No sequence found with id 439
No sequence found with id 440
No sequence found with id 443
No sequence found with id 445
No sequence found with id 446
No sequence found with id 447
No sequence found with id 449
No sequence found with id 454
No sequence found with id 456
No sequence found with id 463
No sequence found with id 481
No sequence found with id 482
No sequence found with id 484
No sequence found with id 488
No sequence found with id 492
No sequence found with id 493
No sequence found with id 494
No sequence found with id 496
No sequence found with id 498
No sequence found with id 507
No sequence found with id 508
No sequence found with id 510
No sequence found with id 516
No sequence found with id 517
No sequence found with id 518
No sequence found with id 519
No sequence found with id 521
No sequence found with id 523
No sequence found with id 531
No sequence found with id 533
No sequence found with id 542
No sequence found with id 543
No sequence found with id 558
No sequence found with id 566
No sequence found with id 567
No sequence found with id 568
No sequence found with id 570
No sequence found with id 571
No sequence found with id 572
No sequence found with id 577
No sequence found with id 585
No sequence found with id 586
No sequence found with id 588
No sequence found with id 589
No sequence found with id 596
No sequence found with id 600
No sequence found with id 606
No sequence found with id 608
No sequence found with id 622
No sequence found with id 624
```

```
No sequence found with id 658
No sequence found with id 668
No sequence found with id 675
No sequence found with id 692
No sequence found with id 706
No sequence found with id 707
No sequence found with id 715
No sequence found with id 717
No sequence found with id 737
No sequence found with id 738
No sequence found with id 739
No sequence found with id 740
No sequence found with id 748
No sequence found with id 753
No sequence found with id 758
No sequence found with id 766
No sequence found with id 767
No sequence found with id 768
No sequence found with id 769
No sequence found with id 773
No sequence found with id 774
No sequence found with id 775
No sequence found with id 779
No sequence found with id 781
No sequence found with id 787
No sequence found with id 789
No sequence found with id 797
No sequence found with id 815
No sequence found with id 820
No sequence found with id 821
No sequence found with id 824
No sequence found with id 826
No sequence found with id 829
No sequence found with id 834
No sequence found with id 838
No sequence found with id 843
No sequence found with id 844
No sequence found with id 853
No sequence found with id 855
No sequence found with id 868
No sequence found with id 872
No sequence found with id 879
No sequence found with id 881
No sequence found with id 887
No sequence found with id 904
No sequence found with id 909
No sequence found with id 910
No sequence found with id 911
No sequence found with id 914
No sequence found with id 919
No sequence found with id 926
No sequence found with id 927
No sequence found with id 931
No sequence found with id 933
No sequence found with id 936
No sequence found with id 960
No sequence found with id 962
No sequence found with id 971
No sequence found with id 972
No sequence found with id 973
```

```
No sequence found with id 980
No sequence found with id 984
No sequence found with id 985
No sequence found with id 986
No sequence found with id 991
No sequence found with id 993
No sequence found with id 1012
No sequence found with id 1017
No sequence found with id 1021
No sequence found with id 1023
No sequence found with id 1024
No sequence found with id 1028
No sequence found with id 1029
No sequence found with id 1031
No sequence found with id 1034
No sequence found with id 1037
No sequence found with id 1040
No sequence found with id 1041
No sequence found with id 1047
No sequence found with id 1064
No sequence found with id 1084
No sequence found with id 1087
No sequence found with id 1094
No sequence found with id 1095
No sequence found with id 1096
No sequence found with id 1113
No sequence found with id 1119
No sequence found with id 1121
No sequence found with id 1123
No sequence found with id 1130
No sequence found with id 1134
No sequence found with id 1139
No sequence found with id 1142
No sequence found with id 1148
No sequence found with id 1151
No sequence found with id 1158
No sequence found with id 1179
No sequence found with id 1197
No sequence found with id 1200
No sequence found with id 1201
No sequence found with id 1207
No sequence found with id 1210
No sequence found with id 1215
No sequence found with id 1225
No sequence found with id 1229
No sequence found with id 1236
No sequence found with id 1237
No sequence found with id 1246
No sequence found with id 1253
No sequence found with id 1259
No sequence found with id 1263
No sequence found with id 1265
No sequence found with id 1266
No sequence found with id 1268
No sequence found with id 1276
No sequence found with id 1290
No sequence found with id 1305
No sequence found with id 1308
No sequence found with id 1317
No sequence found with id 1332
```

```
No sequence found with id 1333
No sequence found with id 1346
No sequence found with id 1351
No sequence found with id 1353
No sequence found with id 1355
No sequence found with id 1365
No sequence found with id 1368
No sequence found with id 1380
No sequence found with id 1383
No sequence found with id 1387
No sequence found with id 1390
No sequence found with id 1398
No sequence found with id 1411
No sequence found with id 1412
No sequence found with id 1414
No sequence found with id 1415
No sequence found with id 1437
No sequence found with id 1444
No sequence found with id 1450
No sequence found with id 1454
No sequence found with id 1456
No sequence found with id 1457
No sequence found with id 1459
No sequence found with id 1460
No sequence found with id 1462
No sequence found with id 1467
No sequence found with id 1469
No sequence found with id 1473
No sequence found with id 1498
No sequence found with id 1501
No sequence found with id 1504
No sequence found with id 1514
No sequence found with id 1515
No sequence found with id 1516
No sequence found with id 1519
No sequence found with id 1527
No sequence found with id 1533
No sequence found with id 1537
No sequence found with id 1541
No sequence found with id 1542
No sequence found with id 1550
No sequence found with id 1561
No sequence found with id 1562
No sequence found with id 1565
No sequence found with id 1576
No sequence found with id 1588
No sequence found with id 1591
No sequence found with id 1592
No sequence found with id 1604
No sequence found with id 1608
No sequence found with id 1610
No sequence found with id 1611
No sequence found with id 1619
No sequence found with id 1634
No sequence found with id 1635
No sequence found with id 1636
No sequence found with id 1638
No sequence found with id 1642
No sequence found with id 1660
No sequence found with id 1661
```

```
No sequence found with id 1662
No sequence found with id 1664
No sequence found with id 1684
No sequence found with id 1687
No sequence found with id 1701
No sequence found with id 1710
No sequence found with id 1717
No sequence found with id 1719
No sequence found with id 1723
No sequence found with id 1733
No sequence found with id 1735
No sequence found with id 1740
No sequence found with id 1748
No sequence found with id 1749
No sequence found with id 1759
No sequence found with id 1765
No sequence found with id 1766
No sequence found with id 1771
No sequence found with id 1784
No sequence found with id 1785
No sequence found with id 1792
No sequence found with id 1794
No sequence found with id 1804
No sequence found with id 1807
No sequence found with id 1817
No sequence found with id 1828
No sequence found with id 1832
No sequence found with id 1838
No sequence found with id 1843
No sequence found with id 1852
No sequence found with id 1855
No sequence found with id 1859
No sequence found with id 1867
No sequence found with id 1882
No sequence found with id 1883
No sequence found with id 1887
No sequence found with id 1888
No sequence found with id 1905
No sequence found with id 1906
No sequence found with id 1919
No sequence found with id 1929
No sequence found with id 1931
No sequence found with id 1932
No sequence found with id 1933
No sequence found with id 1938
No sequence found with id 1939
No sequence found with id 1944
No sequence found with id 1949
No sequence found with id 1951
No sequence found with id 1952
No sequence found with id 1976
No sequence found with id 1980
No sequence found with id 1982
No sequence found with id 2000
No sequence found with id 2006
No sequence found with id 2008
No sequence found with id 2009
No sequence found with id 2010
No sequence found with id 2012
No sequence found with id 2016
```

```
No sequence found with id 2021
No sequence found with id 2022
No sequence found with id 2024
No sequence found with id 2025
No sequence found with id 2026
No sequence found with id 2034
No sequence found with id 2051
No sequence found with id 2060
No sequence found with id 2069
No sequence found with id 2077
No sequence found with id 2088
No sequence found with id 2089
No sequence found with id 2090
No sequence found with id 2093
No sequence found with id 2094
No sequence found with id 2105
No sequence found with id 2121
No sequence found with id 2124
No sequence found with id 2132
No sequence found with id 2133
No sequence found with id 2134
No sequence found with id 2135
No sequence found with id 2141
No sequence found with id 2142
No sequence found with id 2143
No sequence found with id 2154
No sequence found with id 2166
No sequence found with id 2171
No sequence found with id 2172
No sequence found with id 2178
No sequence found with id 2181
No sequence found with id 2188
No sequence found with id 2199
No sequence found with id 2201
No sequence found with id 2203
No sequence found with id 2208
No sequence found with id 2211
No sequence found with id 2212
No sequence found with id 2221
No sequence found with id 2227
No sequence found with id 2231
No sequence found with id 2232
No sequence found with id 2233
No sequence found with id 2234
No sequence found with id 2240
No sequence found with id 2250
No sequence found with id 2252
No sequence found with id 2253
No sequence found with id 2263
No sequence found with id 2264
No sequence found with id 2278
No sequence found with id 2288
No sequence found with id 2293
No sequence found with id 2299
No sequence found with id 2300
No sequence found with id 2304
No sequence found with id 2309
No sequence found with id 2311
No sequence found with id 2317
No sequence found with id 2320
```

```
No sequence found with id 2321
No sequence found with id 2323
No sequence found with id 2324
No sequence found with id 2329
No sequence found with id 2353
No sequence found with id 2367
No sequence found with id 2368
No sequence found with id 2372
No sequence found with id 2376
No sequence found with id 2387
No sequence found with id 2391
No sequence found with id 2392
No sequence found with id 2395
No sequence found with id 2403
No sequence found with id 2410
No sequence found with id 2416
No sequence found with id 2423
No sequence found with id 2430
No sequence found with id 2431
No sequence found with id 2435
No sequence found with id 2439
No sequence found with id 2446
No sequence found with id 2462
No sequence found with id 2475
No sequence found with id 2480
No sequence found with id 2490
No sequence found with id 2494
No sequence found with id 2495
No sequence found with id 2500
No sequence found with id 2514
No sequence found with id 2515
No sequence found with id 2529
No sequence found with id 2543
No sequence found with id 2553
No sequence found with id 2564
No sequence found with id 2572
No sequence found with id 2579
No sequence found with id 2593
No sequence found with id 10
No sequence found with id 38
No sequence found with id 40
No sequence found with id 42
No sequence found with id 45
No sequence found with id 46
No sequence found with id 49
No sequence found with id 55
No sequence found with id 60
No sequence found with id 74
No sequence found with id 77
No sequence found with id 78
No sequence found with id 81
No sequence found with id 90
No sequence found with id 99
No sequence found with id 118
No sequence found with id 124
No sequence found with id 134
No sequence found with id 153
No sequence found with id 156
No sequence found with id 164
No sequence found with id 165
```

```
No sequence found with id 187
No sequence found with id 208
No sequence found with id 209
No sequence found with id 212
No sequence found with id 213
No sequence found with id 221
No sequence found with id 222
No sequence found with id 226
No sequence found with id 228
No sequence found with id 229
No sequence found with id 230
No sequence found with id 238
No sequence found with id 239
No sequence found with id 241
No sequence found with id 249
No sequence found with id 257
No sequence found with id 270
No sequence found with id 272
No sequence found with id 273
No sequence found with id 274
No sequence found with id 275
No sequence found with id 278
No sequence found with id 291
No sequence found with id 300
No sequence found with id 302
No sequence found with id 304
No sequence found with id 307
No sequence found with id 319
No sequence found with id 321
No sequence found with id 361
No sequence found with id 367
No sequence found with id 369
No sequence found with id 373
No sequence found with id 377
No sequence found with id 378
No sequence found with id 383
No sequence found with id 386
No sequence found with id 390
No sequence found with id 402
No sequence found with id 404
No sequence found with id 407
No sequence found with id 419
No sequence found with id 421
No sequence found with id 422
No sequence found with id 423
No sequence found with id 424
No sequence found with id 427
No sequence found with id 432
No sequence found with id 439
No sequence found with id 440
No sequence found with id 443
No sequence found with id 445
No sequence found with id 446
No sequence found with id 447
No sequence found with id 449
No sequence found with id 454
No sequence found with id 456
No sequence found with id 463
No sequence found with id 481
No sequence found with id 482
```

```
No sequence found with id 484
No sequence found with id 488
No sequence found with id 492
No sequence found with id 493
No sequence found with id 494
No sequence found with id 496
No sequence found with id 498
No sequence found with id 507
No sequence found with id 508
No sequence found with id 510
No sequence found with id 516
No sequence found with id 517
No sequence found with id 518
No sequence found with id 519
No sequence found with id 521
No sequence found with id 523
No sequence found with id 531
No sequence found with id 533
No sequence found with id 542
No sequence found with id 543
No sequence found with id 558
No sequence found with id 566
No sequence found with id 567
No sequence found with id 568
No sequence found with id 570
No sequence found with id 571
No sequence found with id 572
No sequence found with id 577
No sequence found with id 585
No sequence found with id 586
No sequence found with id 588
No sequence found with id 589
No sequence found with id 596
No sequence found with id 600
No sequence found with id 606
No sequence found with id 608
No sequence found with id 622
No sequence found with id 624
No sequence found with id 658
No sequence found with id 668
No sequence found with id 675
No sequence found with id 692
No sequence found with id 706
No sequence found with id 707
No sequence found with id 715
No sequence found with id 717
No sequence found with id 737
No sequence found with id 738
No sequence found with id 739
No sequence found with id 740
No sequence found with id 748
No sequence found with id 753
No sequence found with id 758
No sequence found with id 766
No sequence found with id 767
No sequence found with id 768
No sequence found with id 769
No sequence found with id 773
No sequence found with id 774
No sequence found with id 775
```

```
No sequence found with id 779
No sequence found with id 781
No sequence found with id 787
No sequence found with id 789
No sequence found with id 797
No sequence found with id 815
No sequence found with id 820
No sequence found with id 821
No sequence found with id 824
No sequence found with id 826
No sequence found with id 829
No sequence found with id 834
No sequence found with id 838
No sequence found with id 843
No sequence found with id 844
No sequence found with id 853
No sequence found with id 855
No sequence found with id 868
No sequence found with id 872
No sequence found with id 879
No sequence found with id 881
No sequence found with id 887
No sequence found with id 904
No sequence found with id 909
No sequence found with id 910
No sequence found with id 911
No sequence found with id 914
No sequence found with id 919
No sequence found with id 926
No sequence found with id 927
No sequence found with id 931
No sequence found with id 933
No sequence found with id 936
No sequence found with id 960
No sequence found with id 962
No sequence found with id 971
No sequence found with id 972
No sequence found with id 973
No sequence found with id 980
No sequence found with id 984
No sequence found with id 985
No sequence found with id 986
No sequence found with id 991
No sequence found with id 993
No sequence found with id 1012
No sequence found with id 1017
No sequence found with id 1021
No sequence found with id 1023
No sequence found with id 1024
No sequence found with id 1028
No sequence found with id 1029
No sequence found with id 1031
No sequence found with id 1034
No sequence found with id 1037
No sequence found with id 1040
No sequence found with id 1041
No sequence found with id 1047
No sequence found with id 1064
No sequence found with id 1084
No sequence found with id 1087
```

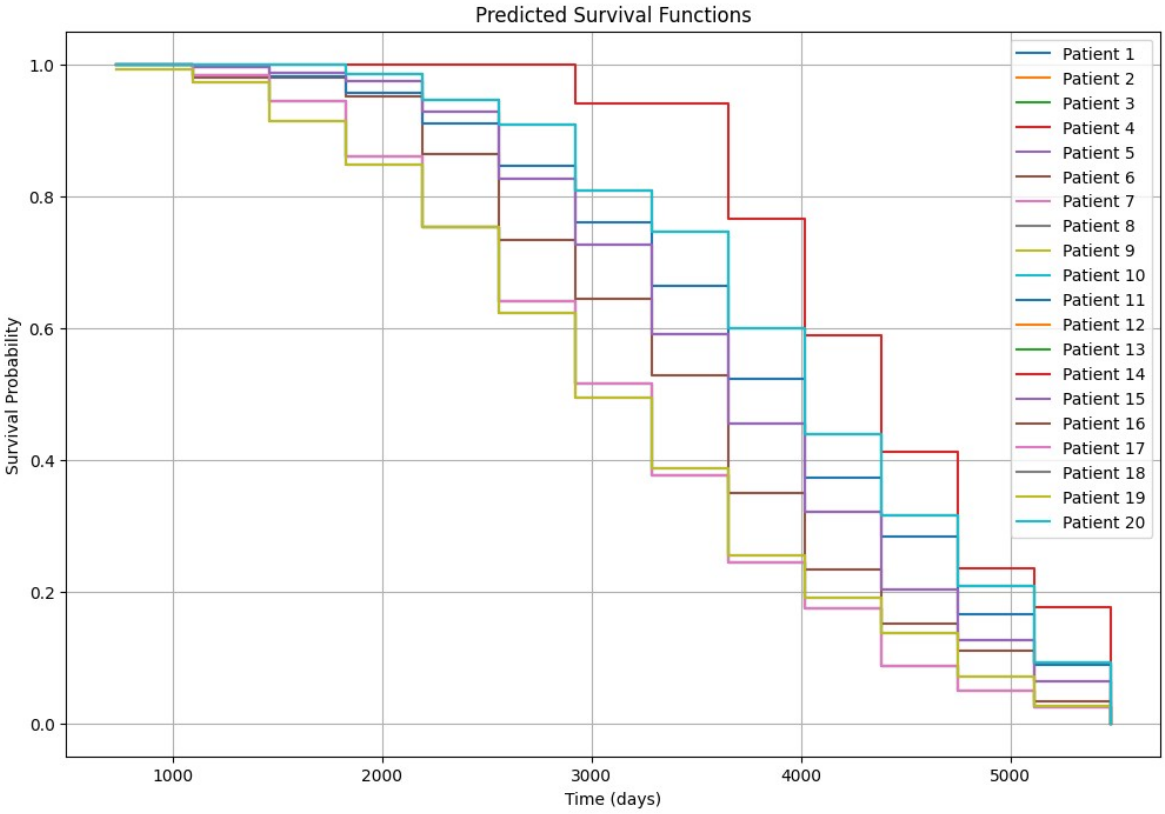
```
No sequence found with id 1094
No sequence found with id 1095
No sequence found with id 1096
No sequence found with id 1113
No sequence found with id 1119
No sequence found with id 1121
No sequence found with id 1123
No sequence found with id 1130
No sequence found with id 1134
No sequence found with id 1139
No sequence found with id 1142
No sequence found with id 1148
No sequence found with id 1151
No sequence found with id 1158
No sequence found with id 1179
No sequence found with id 1197
No sequence found with id 1200
No sequence found with id 1201
No sequence found with id 1207
No sequence found with id 1210
No sequence found with id 1215
No sequence found with id 1225
No sequence found with id 1229
No sequence found with id 1236
No sequence found with id 1237
No sequence found with id 1246
No sequence found with id 1253
No sequence found with id 1259
No sequence found with id 1263
No sequence found with id 1265
No sequence found with id 1266
No sequence found with id 1268
No sequence found with id 1276
No sequence found with id 1290
No sequence found with id 1305
No sequence found with id 1308
No sequence found with id 1317
No sequence found with id 1332
No sequence found with id 1333
No sequence found with id 1346
No sequence found with id 1351
No sequence found with id 1353
No sequence found with id 1355
No sequence found with id 1365
No sequence found with id 1368
No sequence found with id 1380
No sequence found with id 1383
No sequence found with id 1387
No sequence found with id 1390
No sequence found with id 1398
No sequence found with id 1411
No sequence found with id 1412
No sequence found with id 1414
No sequence found with id 1415
No sequence found with id 1437
No sequence found with id 1444
No sequence found with id 1450
No sequence found with id 1454
No sequence found with id 1456
No sequence found with id 1457
```

```
No sequence found with id 1459
No sequence found with id 1460
No sequence found with id 1462
No sequence found with id 1467
No sequence found with id 1469
No sequence found with id 1473
No sequence found with id 1498
No sequence found with id 1501
No sequence found with id 1504
No sequence found with id 1514
No sequence found with id 1515
No sequence found with id 1516
No sequence found with id 1519
No sequence found with id 1527
No sequence found with id 1533
No sequence found with id 1537
No sequence found with id 1541
No sequence found with id 1542
No sequence found with id 1550
No sequence found with id 1561
No sequence found with id 1562
No sequence found with id 1565
No sequence found with id 1576
No sequence found with id 1588
No sequence found with id 1591
No sequence found with id 1592
No sequence found with id 1604
No sequence found with id 1608
No sequence found with id 1610
No sequence found with id 1611
No sequence found with id 1619
No sequence found with id 1634
No sequence found with id 1635
No sequence found with id 1636
No sequence found with id 1638
No sequence found with id 1642
No sequence found with id 1660
No sequence found with id 1661
No sequence found with id 1662
No sequence found with id 1664
No sequence found with id 1684
No sequence found with id 1687
No sequence found with id 1701
No sequence found with id 1710
No sequence found with id 1717
No sequence found with id 1719
No sequence found with id 1723
No sequence found with id 1733
No sequence found with id 1735
No sequence found with id 1740
No sequence found with id 1748
No sequence found with id 1749
No sequence found with id 1759
No sequence found with id 1765
No sequence found with id 1766
No sequence found with id 1771
No sequence found with id 1784
No sequence found with id 1785
No sequence found with id 1792
No sequence found with id 1794
```

```
No sequence found with id 1804
No sequence found with id 1807
No sequence found with id 1817
No sequence found with id 1828
No sequence found with id 1832
No sequence found with id 1838
No sequence found with id 1843
No sequence found with id 1852
No sequence found with id 1855
No sequence found with id 1859
No sequence found with id 1867
No sequence found with id 1882
No sequence found with id 1883
No sequence found with id 1887
No sequence found with id 1888
No sequence found with id 1905
No sequence found with id 1906
No sequence found with id 1919
No sequence found with id 1929
No sequence found with id 1931
No sequence found with id 1932
No sequence found with id 1933
No sequence found with id 1938
No sequence found with id 1939
No sequence found with id 1944
No sequence found with id 1949
No sequence found with id 1951
No sequence found with id 1952
No sequence found with id 1976
No sequence found with id 1980
No sequence found with id 1982
No sequence found with id 2000
No sequence found with id 2006
No sequence found with id 2008
No sequence found with id 2009
No sequence found with id 2010
No sequence found with id 2012
No sequence found with id 2016
No sequence found with id 2021
No sequence found with id 2022
No sequence found with id 2024
No sequence found with id 2025
No sequence found with id 2026
No sequence found with id 2034
No sequence found with id 2051
No sequence found with id 2060
No sequence found with id 2069
No sequence found with id 2077
No sequence found with id 2088
No sequence found with id 2089
No sequence found with id 2090
No sequence found with id 2093
No sequence found with id 2094
No sequence found with id 2105
No sequence found with id 2121
No sequence found with id 2124
No sequence found with id 2132
No sequence found with id 2133
No sequence found with id 2134
No sequence found with id 2135
```

```
No sequence found with id 2141
No sequence found with id 2142
No sequence found with id 2143
No sequence found with id 2154
No sequence found with id 2166
No sequence found with id 2171
No sequence found with id 2172
No sequence found with id 2178
No sequence found with id 2181
No sequence found with id 2188
No sequence found with id 2199
No sequence found with id 2201
No sequence found with id 2203
No sequence found with id 2208
No sequence found with id 2211
No sequence found with id 2212
No sequence found with id 2221
No sequence found with id 2227
No sequence found with id 2231
No sequence found with id 2232
No sequence found with id 2233
No sequence found with id 2234
No sequence found with id 2240
No sequence found with id 2250
No sequence found with id 2252
No sequence found with id 2253
No sequence found with id 2263
No sequence found with id 2264
No sequence found with id 2278
No sequence found with id 2288
No sequence found with id 2293
No sequence found with id 2299
No sequence found with id 2300
No sequence found with id 2304
No sequence found with id 2309
No sequence found with id 2311
No sequence found with id 2317
No sequence found with id 2320
No sequence found with id 2321
No sequence found with id 2323
No sequence found with id 2324
No sequence found with id 2329
No sequence found with id 2353
No sequence found with id 2367
No sequence found with id 2368
No sequence found with id 2372
No sequence found with id 2376
No sequence found with id 2387
No sequence found with id 2391
No sequence found with id 2392
No sequence found with id 2395
No sequence found with id 2403
No sequence found with id 2410
No sequence found with id 2416
No sequence found with id 2423
No sequence found with id 2430
No sequence found with id 2431
No sequence found with id 2435
No sequence found with id 2439
No sequence found with id 2446
```

```
No sequence found with id 2462
No sequence found with id 2475
No sequence found with id 2480
No sequence found with id 2490
No sequence found with id 2494
No sequence found with id 2495
No sequence found with id 2500
No sequence found with id 2514
No sequence found with id 2515
No sequence found with id 2529
No sequence found with id 2543
No sequence found with id 2553
No sequence found with id 2564
No sequence found with id 2572
No sequence found with id 2579
No sequence found with id 2593
```



Les objets construits sont compatible avec la librairie `scikit-surv` et donc permet de réaliser par la suite d'autres analyses.

9. Autres possibilités

Rank : sélectionner les séquences ou entités par rapport à leur rang / indices

La librairie propose un autre critère de sélections qui ne s'appplique qu'aux séquences d'évènements et d'intervalles et qui permet de sélectionner les entités par rapport à leur rang et leur indice.

Exemple : 3 premiers indices (méthodes et critère)

In [47]:
```
# Génération d'une séquences d'évènements
simple_settings = {
    "id_column": "id",
    "time_column": "date",
    "entity_features": ["event_type"],
}
simple_pool = EventSequencePool(event_data, simple_settings)

# Filtre avec une méthode
simple_pool.head(3)

# Filtre avec RankCriterion
rank_criterion = RankCriterion(first=3)
filtered_pool = simple_pool.filter(rank_criterion, level="entity")
```

Séquences d'intervalles et séquences d'évènements

`TanaT` propose directement de travailler avec des séquences d'intervalles et d'évènements avec les objets `IntervaleSequencePool` et `EventSequencePool` . La grande majorité des méthodes proposées pour les séquences d'états sont utilisables pour les séquences d'intervalles et d'évènement et inversement et les fonctions et méthodes sont les mêmes.

Nous souhaiterions voir apparaitre une certaine logique naturelle dans les dissimilarités entre séquences, c'est-à-dire avoir des distances non-nulles :

- si le type d'évènement ou d'intervalle est modifié,
- si le moment où a lieu l'évènement ou l'intervalle est modifié (décalage temporel),
- si la durée de l'intervalle est différente.

Cependant, ce n'est pas ce que `TanaT` propose. Nous l'illustrons ci-après.

Génération de séquences d'évènements et d'intervalles

Séquences d'évènements

In [48]:
```
av26 = dt.date.fromisoformat('2025-04-26')
av27 = dt.date.fromisoformat('2025-04-27')

df_event = pd.DataFrame(
    {
        "id":["reference","decalage_temp","chgt_type","ensemble"],
        "date":[av26,av27,av26,av27],
        "event":["A","A","B","B"]
    }
)

event_settings = {
    "id_column": "id",
    "time_column": "date",
    "entity_features": ["event"],
}

event_pool = EventSequencePool(df_event, event_settings)
```

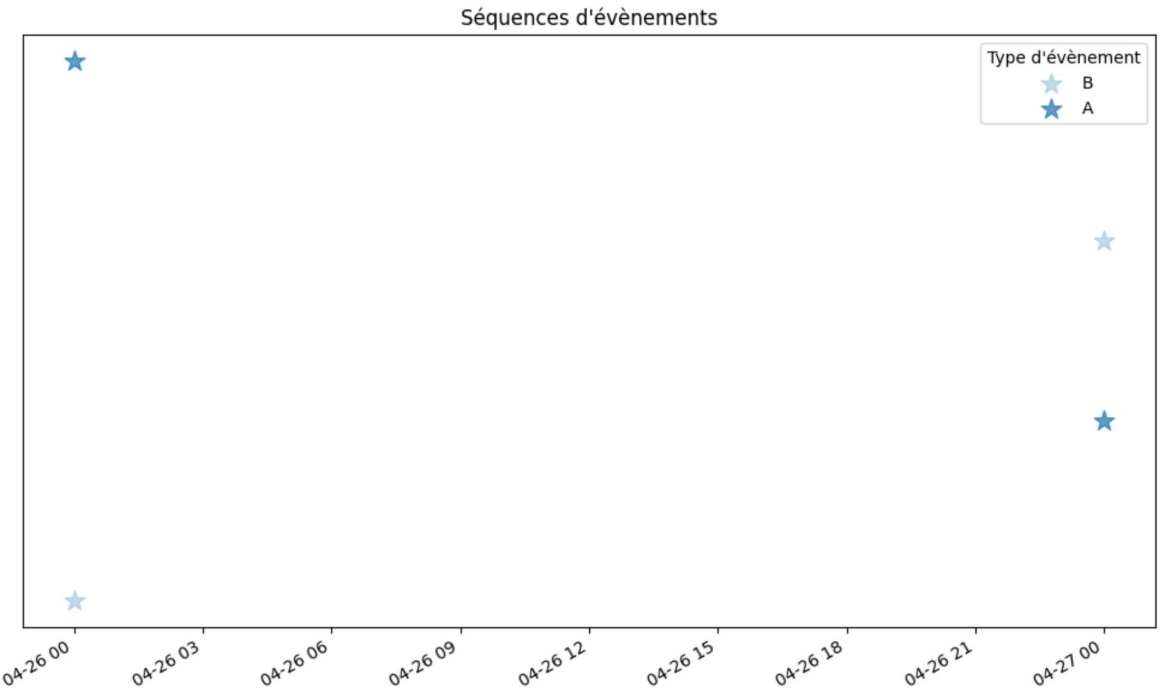
```
event_pool
```

Out[48]:

```
┌──────────────────────────────────────────────────┐
│            EventSequencePool summary            │
└──────────────────────────────────────────────────┘

STATISTICS
──────────────────────────

  Total sequences    4
  Average length     1.0
  Minimum length     1
  Maximum length     1
  Vocabulary size    2

DATA PREVIEW
──────────────────────────

  Sequence Data:
               id       date event
        chgt_type 2025-04-26     B
    decalage_temp 2025-04-27     A
         ensemble 2025-04-27     B

  Static Data:
    None

  Metadata:
    Temporal:
      Type: datetime
      Granularity: DAY

    Entity Features (1):
      - event: categorical

    Static Features (0):
      (none)
```

In [49]:

```
SequenceVisualizer.timeline(stacking_mode="flat") \
    .title("Séquences d'évènements") \
    .colors("Paired") \
    .marker(spacing=0.8, size=8) \
    .legend(show=True, loc="upper right", title="Type d'évènement") \
    .draw(event_pool)
```

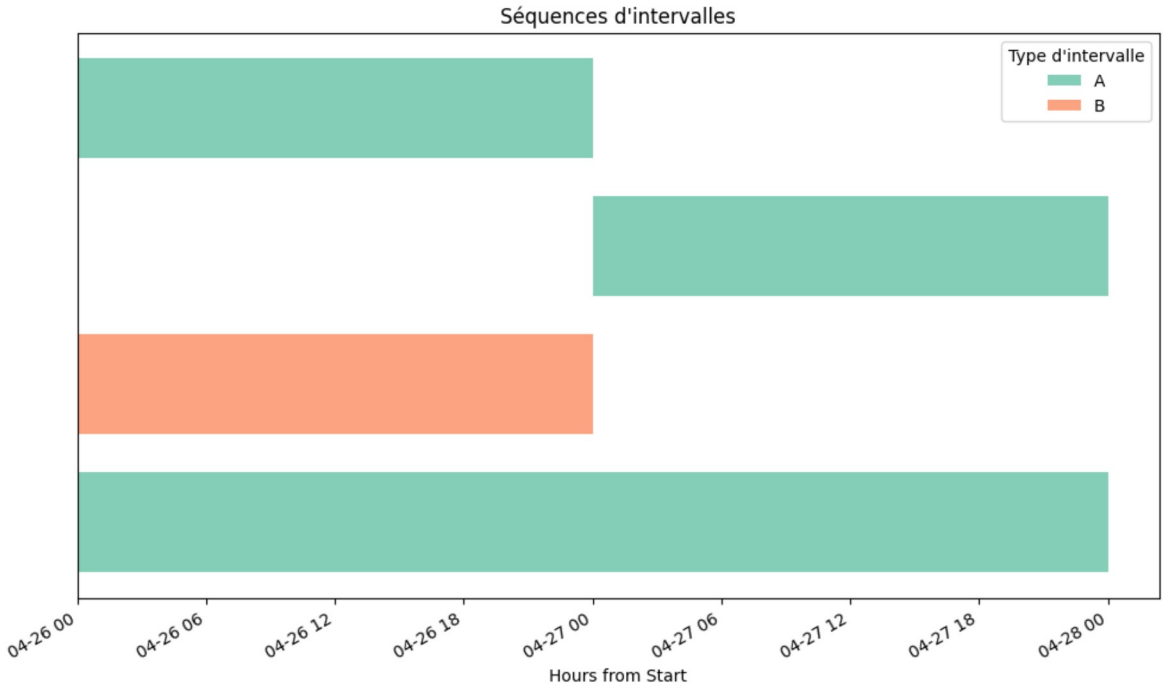


Out[49]:

Séquences d'intervalles

In [50]:
```
av26 = dt.date.fromisoformat('2025-04-26')
av27 = dt.date.fromisoformat('2025-04-27')
av28 = dt.date.fromisoformat('2025-04-28')

df_intervalle = pd.DataFrame(
    {
        "id":["reference","decalage_temp","chgt_type","chgt_duree"],
        "start_date":[av26,av27,av26,av26],
        "end_date":[av27,av28,av27,av28],
        "event":["A","A","B","A"]
    }
)

# Create interval sequence pool
interval_settings = IntervalSequenceSettings(
    id_column="id",
    start_column="start_date",
    end_column="end_date",
    entity_features=["event"],
)

interval_pool = IntervalSequencePool(df_intervalle, interval_settings)
```

In [51]:
```
SequenceVisualizer.timeline(
    granularity="day",
    stacking_mode="flat"
) \
    .title("Séquences d'intervalles") \
    .colors("Set2") \
    .marker(spacing=0.9, alpha=0.8) \
    .legend(show=True, title="Type d'intervalle", loc="upper right") \
    .x_axis(label="Hours from Start") \
    .draw(interval_pool)
```



Out[51]:

In [52]:
```
# Chi2
settings = Chi2SequenceMetricSettings()
chi2_metric = Chi2SequenceMetric(settings=settings)

# DTW
settings = DTWSequenceMetricSettings()
dtw_metric = DTWSequenceMetric(settings=settings)


# Soft-DTW
settings = SoftDTWSequenceMetricSettings()
soft_dtw_metric = SoftDTWSequenceMetric(settings=settings)


# Linear Pariwise
settings = LinearPairwiseSequenceMetricSettings()
linear_pairwise_metric = LinearPairwiseSequenceMetric(settings=settings)


# OM
settings = EditSequenceMetricSettings()
edit_metric = EditSequenceMetric(settings=settings)


# LCP (+ grand score = meilleure ressemblance)
settings = LCPSequenceMetricSettings()
lcp_metric = LCPSequenceMetric(settings=settings)


# LCS (+ grand score = meilleure ressemblance)
settings = LCSSequenceMetricSettings()
lcs_metric = LCSSequenceMetric(settings=settings)
```

Distances sur les évènements

In [53]:
```
pool = event_pool

results = {
    "Chi2": chi2_metric.compute_matrix(pool, window=3),
    "DTW": dtw_metric.compute_matrix(pool, window=3),
    "Soft-DTW": soft_dtw_metric.compute_matrix(pool),
    "Linear_Pairwise": linear_pairwise_metric.compute_matrix(pool),
    "OM": edit_metric.compute_matrix(pool),
    "LCP": lcp_metric.compute_matrix(pool),
    "LCS": lcs_metric.compute_matrix(pool)
}

df_metrics_event = pd.concat(
    {name: dm.to_dataframe()["reference"] for name, dm in results.items()},
    axis=1
)
```

```
┌─ Chi2SequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:00<00:00,  6.32it/s]
│
└─ Done (4x4 matrix, 0.95s)
┌─ DTWSequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:01<00:00,  5.13it/s]
│
└─ Done (4x4 matrix, 1.38s)
┌─ SoftDTWSequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:01<00:00,  3.83it/s]
│
└─ Done (4x4 matrix, 1.57s)
┌─ LinearPairwiseSequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:00<00:00, 13617.87it/s]
│
└─ Done (4x4 matrix, 0.00s)
┌─ EditSequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:00<00:00, 33599.23it/s]
│
└─ Done (4x4 matrix, 0.00s)
┌─ LCPSequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:00<00:00, 11.90it/s]
│
└─ Done (4x4 matrix, 0.51s)
┌─ LCSSequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:00<00:00,  6.50it/s]
│
└─ Done (4x4 matrix, 0.93s)
```

In [54]:
```
print(df_metrics_event)
```

	Chi2	DTW	Soft-DTW	Linear_Pairwise	OM	LCP	LCS
chgt_type	1.414214	1.0	1.0	1.0	1.0	0.0	0.0
decalage_temp	0.000000	0.0	0.0	0.0	0.0	1.0	1.0
ensemble	1.414214	1.0	1.0	1.0	1.0	0.0	0.0
reference	0.000000	0.0	0.0	0.0	0.0	1.0	1.0

Distances sur les intervalles

In [55]:
```
pool = interval_pool

results = {
    "Chi2": chi2_metric.compute_matrix(pool, window=3),
    "DTW": dtw_metric.compute_matrix(pool, window=3),
    "Soft-DTW": soft_dtw_metric.compute_matrix(pool),
    "Linear_Pairwise": linear_pairwise_metric.compute_matrix(pool),
    "OM": edit_metric.compute_matrix(pool),
    "LCP": lcp_metric.compute_matrix(pool),
    "LCS": lcs_metric.compute_matrix(pool)
}

df_metrics_interval = pd.concat(
    {name: dm.to_dataframe()["reference"] for name, dm in results.items()},
    axis=1
)
```

```
┌─ Chi2SequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:00<00:00, 21363.18it/s]
│
└─ Done (4x4 matrix, 0.01s)
┌─ DTWSequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:00<00:00, 29746.84it/s]
│
└─ Done (4x4 matrix, 0.01s)
┌─ SoftDTWSequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:00<00:00, 26658.71it/s]
│
└─ Done (4x4 matrix, 0.00s)
┌─ LinearPairwiseSequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:00<00:00, 42871.93it/s]
│
└─ Done (4x4 matrix, 0.00s)
┌─ EditSequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:00<00:00, 35001.15it/s]
│
└─ Done (4x4 matrix, 0.00s)
┌─ LCPSequenceMetric
│
│ Progress: 100%|████████| 6/6 [00:00<00:00, 12826.62it/s]
│
└─ Done (4x4 matrix, 0.00s)
┌─ LCSSequenceMetric
│
```

```
Progress: 100%|████████| 6/6 [00:00<00:00, 32058.37it/s]
└─ Done (4x4 matrix, 0.00s)
```

In [56]:
```
print(df_metrics_interval)
```

```
                  Chi2  DTW  Soft-DTW  Linear_Pairwise   OM  LCP  LCS
chgt_duree    2.828427  0.0       0.0              0.0  0.0  1.0  1.0
chgt_type     6.928203  1.0       1.0              1.0  1.0  0.0  0.0
decalage_temp 0.000000  0.0       0.0              0.0  0.0  1.0  1.0
reference     0.000000  0.0       0.0              0.0  0.0  1.0  1.0
```

Pour les séquences d'intervalles et d'évènements, les distances ne fonctionnent pas comme on pourrait s'y attendre avec une prise en compte du décalage temporel et du changement de durée plus que le nombre d'intervalles ou d'évènements et les changements de types. A l'exception de la distance χ^2 pour les séquences d'intervalles (car c'est une distance de distribution et donc prend en compte le temps passé dans chaque type d'intervalle), les métriques ne prennent jamais en compte ces aspects et ramènent finalement ces deux types de séquences à des séquences d'états sans que la temporalité n'ai de réel impact si ce n'est dans l'ordre dans lequel les évènements et intervalles ont lieu.